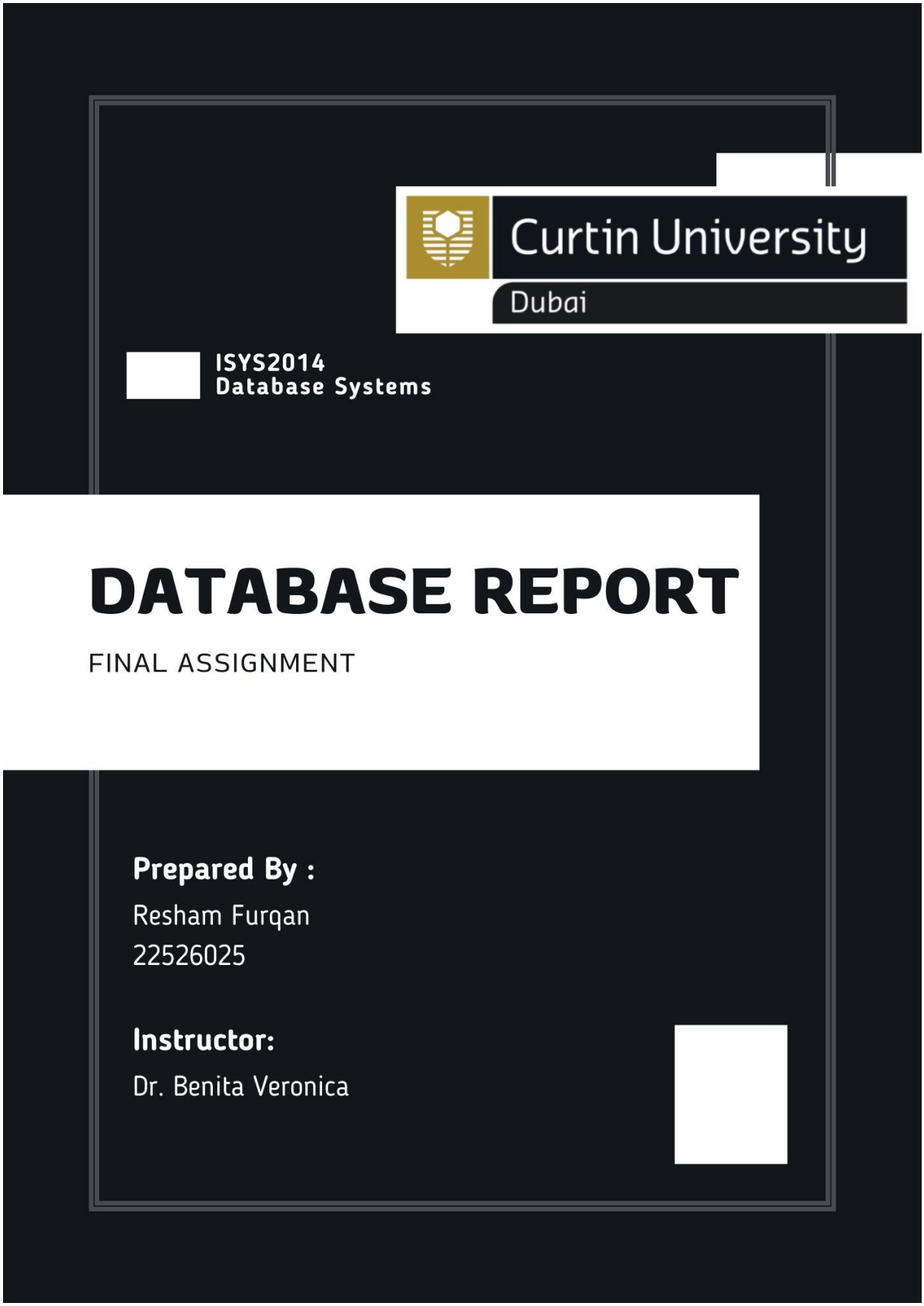

Curtin University
Dubai
ISYS2014
Database Systems
DATABASE REPORT
FINAL ASSIGNMENT
Prepared By :
Resham Furqan
22526025
Instructor:
Dr. Benita Veronica

Table of Contents

1. Introduction 3
2. Scenario Overview 3
3. Database Design 4
   - 3.1 Selection and Justification of Entities and Relationships 4
   - 3.2 Referential Integrity Constraints 8
   - 3.3 Entity-Relationship (ER) Diagram 10
   - 3.4 Relational Schema 11
   - 3.5 Data Description and Justification 12
   - 3.6 Assumptions and Business Rules 20
4. Database Implementation 23
   - 4.1 Implementation Process 23
   - 4.2 Sample Data and Data Sources 24
   - 4.3 Evidence of Implementation 25
5. Query Design and Results 29
   - 5.1 Query Requirements and Objectives 29
   - 5.2 SQL Query Implementations 29
   - 5.3 Sample Outputs 32
6. Advanced Database Features 36
   - 6.1 Stored Procedures 36
   - 6.2 Triggers 38
   - 6.3 Views and Indexes 39
   - 6.4 Evidence of Advanced Feature Implementation 41
7. Database Connectivity 52
   - 7.1 Programming Language Interface (Python) 52
   - 7.2 Code Implementation and Results 57
8. Discussion 61
   - 8.1 Achievements and Reflections 61
   - 8.2 Challenges and Limitations 62
   - 8.3 Suggestions for Improvement 64
9. References 65

1. Introduction

This report demonstrates the practical application of database design, implementation, and querying a real-world database system using the knowledge gained in this unit. It focuses on building a relational database for managing the information relating to the global film festivals, including awards, participations, events, nominations, and winners. The report includes the ER Diagram, conversion to relational schema, implementation of the database in MySQL, execution of SQL queries, integration of triggers and stored procedures, and the connection of database with a Python program to retrieve and manipulate data.

2. Scenario Overview

The assignment covers the information management challenges related to the film festivals happening all around the world, some are country-specific while others are held internationally to showcase the cinematic production and celebrate the achievements of individuals and the film industry.

The database system will be designed to store and organise the important information and certain details about these events and people involved in them to efficiently handle the data of various festivals across different years to enable useful queries to help the people interested in discovering the global film festival activities.

3. Database Design

3.1 Selection and Justification of Entities and Relationships

Entities and their Attributes:

Entity Sets	Primary Keys	Other Attributes
film_festival	festival_ID	name, website, country, location, founding_year, festival_type
film	film_ID	title, release_year, language, original_title, duration
nomination	nomination_ID	is_unanimous, status (nominated/won)
edition	edition_ID	venue, start_date, end_date, year
country	country_code	region, country_name
person	person_ID	first_name, last_name, dob, profession, nationality, gender
award_category	category_ID	category_name, level (main/secondary), description

Relationship Sets, their Participating Entities, Attributes of Relationship Sets (if any), Cardinality and Participation Constraints:

Relationship Sets	Participating Entities	Attributes	Cardinality	Participation
hosts	film_festival and edition	-	1:N One-to-Many	Partial on film_festival and Total on edition
awards	award_category and nomination	-	1:N One-to-Many	Partial on both
belongs_to	person and country	-	N:1 Many-to-One	Total on person and Partial on country
receives	film and nomination	-	1:N One-to-Many	Partial on both
produced_in	film and country	production_role	M:N Many-to-Many	Partial on both
has	edition and award_category	-	1:N One-to-Many	Partial on both

creates	person and film	role	M:N Many-to-Many	Partial on both

Design Justification:

Core Entities:

1. film_festival:

It is the central entity that represents the film festivals to support the given scenario's requirement for keeping the record of multiple festivals worldwide.

2. film:

It is significant for recording the award-winning films and the nominations from the provided data.

3. edition:

It is a weak entity, and its existence depends on the owner entity, 'film_festival'. This is shown in the ER Diagram with a doubled-lined rectangle for 'edition' entity and double-lined diamond for the identifying relationship, 'hosts'. It uses a surrogate primary key (edition_ID) for uniquely identifying each festival edition and to simplify joins. It also has a foreign key (festival_ID) which is linked to 'film_festival', which allows the tracking of historical data across years including the current year as it specified.

4. person:

It captures the directors, technical people, and actors mentioned in the scenario and data sources.

5. award_category:

It models the various award types, such as Best Actor, Best Producer etc supporting the diverse award scenarios.

Relationship Design:

1. M:N Relationships:

Many-to-Many relationships handles the complex real-world scenarios where films have multiple countries and crew members. The relationship tables ‘film_crew’ and ‘film_production’ are created to resolve the M:N relationships ‘creates’ and ‘produced_in’ respectively.

2. Associative Entity:

nomination is an associative entity which flexibly handles both the nominations and the wins with status attribute.

3. Weak Entity:

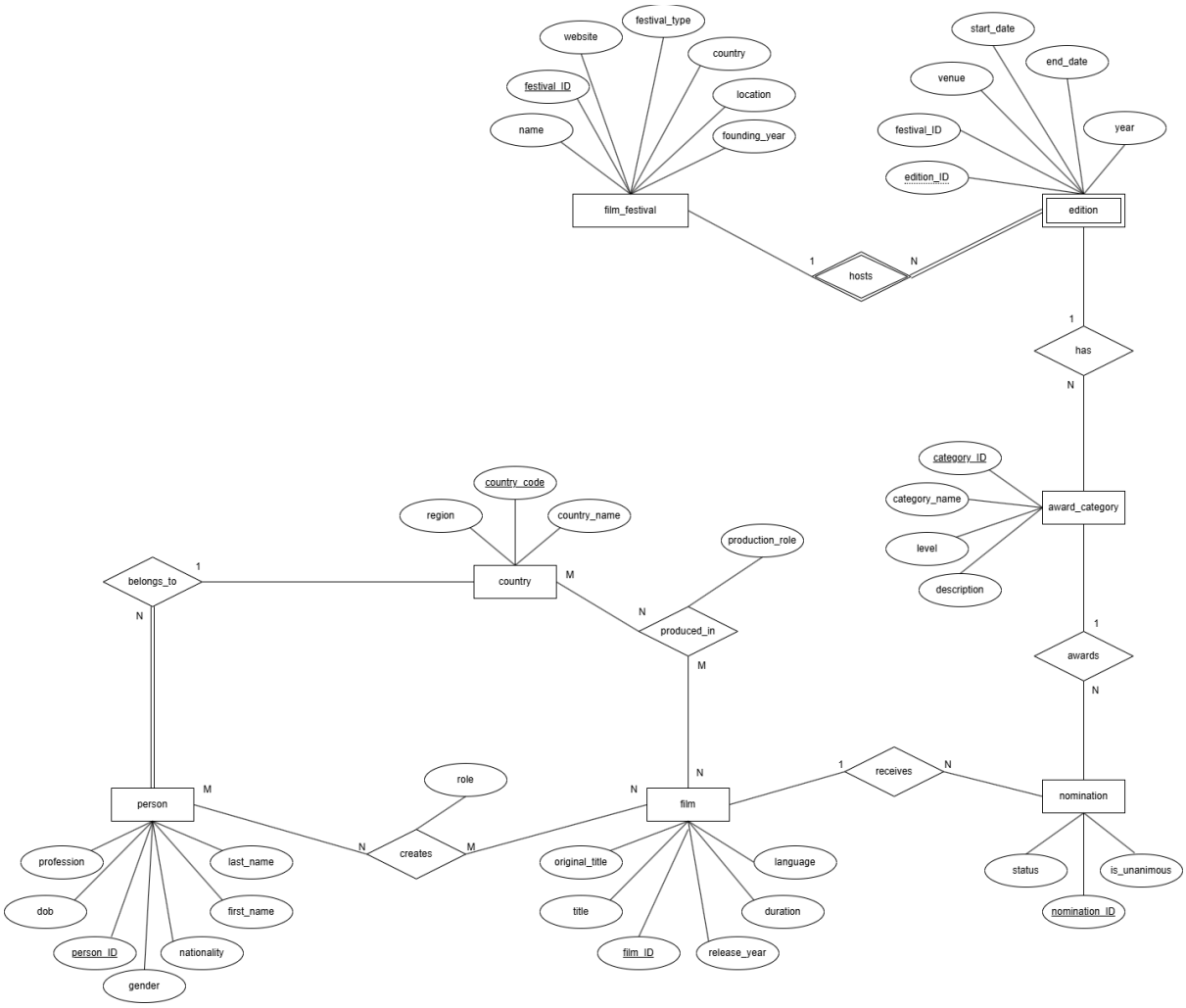
‘edition’ is considered as a weak entity because it depends on the ‘fim_festival’ entity for its existence.

3.2 Referential Integrity Constraints

Foreign Key	References	Purpose	Relationship Type
nomination.film_ID	film.film_ID	To ensure the nominated films exist	Regular
nomination.edition_ID	edition.edition_ID	To associate the nominations with their specific editions	Regular
nomination.person_ID	person.person_ID	To validate person reference	Regular
nomination.category_ID	award_category.category_ID	To validate award categories	Regular
edition.festival_ID	film_festival.festival_ID	To link the editions to its parent festivals	Weak Entity

film_production.country_code	country.country_code	To implement 'produced_in relationship	M:N Relationship
film_production.film_ID	film.film_ID	To implement 'produced_in' relationship	M:N Relationship
person_nationality	country.country_code	To validate nationality codes	Attribute
film_crew.person_ID	person.person_ID	To implement 'creates' relationship	M:N Relationship
film_crew.film_ID	film.film_ID	To implement 'creates' relationship	M:N Relationship

3.3 Entity-Relationship (ER) Diagram



[3]

3.4 Relational Schema

Primary Tables:

film_festival (festival_ID, name, website, country, location, founding_year, festival_type)

film (film_ID, title, release_year, language, original_title, duration)

nomination (nomination_ID, film_ID, edition_ID, person_ID, category_ID, is_unanimous, status)

edition (edition_ID, festival_ID, venue, start_date, end_date, year)

country (country_code, region, country_name)

person (person_ID, first_name, last_name, dob, profession, nationality, gender)

award_category (category_ID, category_name, level, description)

Relationship Tables:

film_production (film_ID, country_code, production_role)

film_crew (film_ID, person_ID, role)

3.5 Data Description and Justification

Table: film_festival

Attribute	Data Type	Description	Constraints
festival_ID	INT	The unique identifier for each festival	PRIMARY KEY, NOT NULL
name	VARCHAR(100)	The name of the festival	NOT NULL
website	VARCHAR(200)	The official website URL	-
country	VARCHAR(50)	The host country	NOT NULL
location	VARCHAR(100)	The city where the festival is held	NOT NULL
founding_year	INT	The year the festival was established	CHECK (founding_year > 1900)
festival_type	VARCHAR(20)	The type of festival (international, national, etc)	NOT NULL

Table: film

Attribute	Data Type	Description	Constraints
film_ID	INT	The unique identifier for each film	PRIMARY KEY, NOT NULL
title	VARCHAR(200)	English title of the film	NOT NULL
release_year	INT	The year the fil was released in	CHECK (release_year > 1900)
language	VARCHAR(50)	The primary language of the film	-
original_title	VARCHAR(200)	The title in the original language	-
duration	INT	The runtime in minutes	CHECK (duration > 0)

Table: nomination

Attribute	Data Type	Description	Constraints
nomination_ID	INT	The unique identifier for each nomination	PRIMARY KEY, NOT NULL
film_ID	INT	Reference to the nominated film	FOREIGN KEY, NOT NULL
edition_ID	INT	Reference to the festival edition	FOREIGN KEY, NOT NULL
person_ID	INT	Reference to the winning person	FOREIGN KEY
category_ID	INT	Reference to the award category	FOREIGN KEY, NOT NULL
is_unanimous	BOOLEAN	If the win was unanimous (yes or no)	DEFAULT FALSE
status	VARCHAR(20)	Nominated/Won	NOT NULL, CHECK (status IN (‘Nominated’, ‘Won’))

Table: edition

Attribute	Data Type	Description	Constraints
edition_ID	INT	The unique identifier for each edition	PRIMARY KEY, NOT NULL
festival_ID	INT	Reference to the parent festival	FOREIGN KEY, NOT NULL
venue	VARCHAR(100)	the primary venue location	-
start_date	DATE	The starting date of the festival	NOT NULL
end_date	DATE	The ending date of the festival	NOT NULL
year	INT	The year of the edition	NOT NULL, CHECK (year > 1900)

Table: country

Attribute	Data Type	Description	Constraints
country_code	CHAR(3)	ISO 3-letter country code	PRIMARY KEY, NOT NULL
region	VARCHAR(50)	The geographical region	-
country_name	VARCHAR(100)	The actual country name	NOT NULL

Table: person

Attribute	Data Type	Description	Constraints
person_ID	INT	The unique identifier for each person	PRIMARY KEY, NOT NULL
first_name	VARCHAR(50)	The first name of the person	NOT NULL
last_name	VARCHAR(50)	The last name of the person	NOT NULL
dob	DATE	The date of birth of the person	-
profession	VARCHAR(100)	The primary profession of the person	-
nationality	VARCHAR(50)	The nationality of the person	-
gender	VARCHAR(10)	To determine the person's gender	NOT NULL

Table: award_category

Attribute	Data Type	Description	Constraint
category_ID	INT	The unique identifier for each category	PRIMARY KEY, NOT NULL
category_name	VARCHAR(100)	The name of the award category	NOT NULL
level	VARCHAR(20)	Description of the award	CHECK (level IN ('Main', 'Secondary', 'Special'))
description	TEXT	Main/Secondary/Special classification	-

Table: film_crew

Attribute	Data Type	Description	Constraints
film_ID	INT	Reference to the film	FOREIGN KEY, Composite PK, NOT NULL
person_ID	INT	Reference to the crew person	FOREIGN KEY, Composite PK, NOT NULL
role	VARCHAR(50)	Role in the film (producer, actor, etc)	NOT NULL

Table: film_production

Attribute	Data Type	Description	Constraints
film_ID	INT	Reference to the film	FOREIGN KEY, Composite PK, NOT NULL
country_code	CHAR(3)	Reference to the production country	FOREIGN KEY, Composite Key, NOT NULL
production_role	VARCHAR(50)	The type of production involvement	NOT NULL

Design Justification:

Data Type Selection:

1. **VARCHAR:** is used with appropriate lengths for textual data based on real-world examples from provided data.
2. **BOOLEAN:** is used for unanimous wins.
3. **DATE:** is used for temporal data that enables date-range queries.
4. **INT:** is used for identifiers and years supporting the historical tracking requirement.
5. **CHAR:** is used for country codes using ISO standards for consistency.

Constraint Decisions:

1. **NOT NULL Constraints:** on mandatory attributes ensuring data integrity.
2. **FOREIGN KEY Constraints:** maintaining referential integrity across related tables.
3. **CHECK Constraints:** on years and durations preventing invalid data.
4. **Unique Constraints:** implicitly through primary keys.

3.6 Assumptions and Business Rules

Business Logic Assumptions:

1. Award Flexibility:

The design supposes that awards can be given to either individuals or the films, with the 'person_ID' in 'nomination' being optional to carefully handle both the scenarios. Every nomination must have either a film or a person.

2. Award Recipient Rule:

In this design, every nomination must have either a film or a person, but not necessarily both.

3. Co-production Handling:

The design assumes that films can have various production countries through the 'film_production' table.

4. Edition Uniqueness:

In this design, each festival can only have one edition per year.

5. Award Hierarchy:

The design supposes that the award categories have level (Main, Secondary, Special) to distinguish between major prizes like Palme d'Or and secondary awards.

6. Multiple Roles:

The design assumes that the individuals can have various roles across multiple films (producer, director, actor) recorded in the 'film_crew' relationship.

7. Historical Tracking:

The design supposes that the ‘edition’ entity supports tracking festival history from 1940s to 2025 as indicated in the requirements.

8. Temporal Consistency:

In this design, the edition ‘start_date’ must be before the ‘end_date’.

Scenario-Specific Assumptions:

1. Global Scope:

The design supports both country-specific and international festivals as described in the given scenario.

2. Multiple Year Support:

The design handles the data for multiple years as well as one year only (including the current one) as required in the brief.

3. Industry Coverage:

Records the films, actors, producers, directors and other people involved as outlined in the scenario.

4. Comprehensive Tracking:

The database system tracks the nominations, special awards, and wins.

Data Integrity Assumptions:

1. Film Identification:

To handle films with similar titles through the ‘release_year’ and ‘original_titles’.

2. Country Standards:

The design uses the ISO 3-letter codes for consistent country representation.

3. Person Identification:

Unique identification of individuals despite potential name variations.

4. Temporal Consistency:

Edition years align with actual festival occurrences, handling cancelled years correctly.

5. Weak Entity Implementation:

The ‘edition_ID’ is a weak entity and does not really have a primary key of its own but we use ‘edition_ID’ as a surrogate primary key for simplicity and efficiency.

Normalization Assumptions:

All the tables in this database satisfies Third Normal Form by ensuring there is no transitive dependencies, and non-key attributes depend on non-key attributes. The design ensured there is minimal data redundancy and maintained query efficiency for common film festival queries. There is proper handling of multi-valued attributes through separate tables for relationships.

4. Database Implementation

4.1 Implementation Process

This database was implemented using a structured multi-script methodology in MySQL version 8.0 to ensure modularity in design and the database is easy to manage.

Order of Script Execution:

1. create_tables.sql (it creates the main database and 9 tables)
2. insert_data.sql (it inserts the real-world data in the tables)
3. query1.1.sql – query2.4.sql (these files contain the queries of both levels (discussed in further sections))
4. advanced_features.sql (this was used to add stored procedures, views, triggers, and indexes (discussed in detail in further sections))
5. test_stored_procedures.sql, test_views.sql, test_indexes.sql, test_triggers.sql (these are test files for the advanced features)

Main Features:

1. All tables have FOREIGN KEY, NOT NULL, PRIMARY KEY, and CHECK Constraints.
2. AUTO_INCREMENT was used for surrogate keys.
3. Referential Integrity was implemented through proper relationship mapping.
4. Suitable data types were selected for each attribute in all the tables.
5. Indexes were created on frequently queried columns for better performance.

4.2 Sample Data and Data Sources

Sample Data Composition:

1. 2 major film festivals were used for the data, Academy Awards (Oscars) and Cannes Film Festival.
2. 69 films were used to represent classic and contemporary cinema.
3. 25 countries were included in the data with ISO codes and regions.
4. 20 film productions records were used to show the international co-productions.
5. 25 festival editions were included from 1928 to 2025.
6. 60 film crew relationships exist in the data that link persons to films with their roles.
7. 85 persons were included in the data consisting of actors, directors, and other crew members.
8. 18 award categories were there in the database that cover main, special, and secondary awards.
9. 45 nominations were included with both wins and nominations.

Data Insertion Method:

All the data was inserted into respective tables using INSERT statements with subqueries to handle the auto-generated IDs efficiently. SELECT subqueries were used to resolve relationships during insertion. Constraint validation was added during data loading process. Batch insertion was added for efficiency with proper error handling.

Data Sources:

1. Academy Awards Historical Data, Oscar winners and nominees from 1927 to 2025 [1].
2. Cannes Film Festival Winners, Palme d'Or recipients from 1946 to 2025 [2].

4.3 Evidence of Implementation

Successful Creation of Tables:

```
mysql> SOURCE create_tables.sql;
Query OK, 1 row affected (0.03 sec)

Database changed
Query OK, 0 rows affected (0.25 sec)

Query OK, 0 rows affected (0.12 sec)

Query OK, 0 rows affected (0.14 sec)

Query OK, 0 rows affected (0.14 sec)

Query OK, 0 rows affected (0.15 sec)

Query OK, 0 rows affected (0.17 sec)

Query OK, 0 rows affected (0.22 sec)

Query OK, 0 rows affected (0.17 sec)

Query OK, 0 rows affected (0.14 sec)
```

Verification:

```
mysql> SHOW TABLES;
+---------------------------+
| Tables_in_Furqan_22526025 |
+---------------------------+
| award_category            |
| country                   |
| edition                   |
| film                      |
| film_crew                 |
| film_festival             |
| film_production           |
| nomination                |
| person                    |
+---------------------------+
9 rows in set (0.13 sec)
```

```
mysql> DESC award_category;
+---------------+-------------+------+-----+---------+----------------+
| Field         | Type        | Null | Key | Default | Extra          |
+---------------+-------------+------+-----+---------+----------------+
| category_ID   | int         | NO   | PRI | NULL    | auto_increment |
| category_name | varchar(100) | NO   |     | NULL    |                |
| level         | varchar(20) | NO   |     | NULL    |                |
| description   | text        | YES  |     | NULL    |                |
+---------------+-------------+------+-----+---------+----------------+
4 rows in set (0.14 sec)
```

```
mysql> DESC country;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| country_code | char(3)      | NO   | PRI | NULL    |       |
| region       | varchar(50)  | YES  |     | NULL    |       |
| country_name | varchar(100) | NO   |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
3 rows in set (0.01 sec)
```

```
mysql> DESC edition;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| edition_ID  | int          | NO   | PRI | NULL    | auto_increment |
| festival_ID | int          | NO   | MUL | NULL    |                |
| venue       | varchar(100) | YES  |     | NULL    |                |
| start_date  | date         | NO   |     | NULL    |                |
| end_date    | date         | NO   |     | NULL    |                |
| year        | int          | NO   |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
6 rows in set (0.01 sec)
```

```
mysql> DESC film;
+---------------+-------------+------+-----+---------+----------------+
| Field         | Type        | Null | Key | Default | Extra          |
+---------------+-------------+------+-----+---------+----------------+
| film_ID       | int         | NO   | PRI | NULL    | auto_increment |
| title         | varchar(200)| NO   |     | NULL    |                |
| release_year  | int         | NO   |     | NULL    |                |
| language      | varchar(50) | YES  |     | NULL    |                |
| original_title| varchar(200)| YES  |     | NULL    |                |
| duration      | int         | YES  |     | NULL    |                |
+---------------+-------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

```
mysql> DESC film_crew;
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| film_ID   | int         | NO   | PRI | NULL    |       |
| person_ID | int         | NO   | PRI | NULL    |       |
| role      | varchar(50) | NO   | PRI | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

```
mysql> DESC film_festival;
+--------------+-------------+------+-----+---------+----------------+
| Field        | Type        | Null | Key | Default | Extra          |
+--------------+-------------+------+-----+---------+----------------+
| festival_ID  | int         | NO   | PRI | NULL    | auto_increment |
| name         | varchar(100)| NO   |     | NULL    |                |
| website      | varchar(200)| YES  |     | NULL    |                |
| country      | varchar(50) | NO   |     | NULL    |                |
| location     | varchar(100)| NO   |     | NULL    |                |
| founding_year| int         | NO   |     | NULL    |                |
| festival_type| varchar(20) | NO   |     | NULL    |                |
+--------------+-------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)
```

```
mysql> DESC film_production;
+-----------------+-------------+------+-----+---------+-------+
| Field           | Type        | Null | Key | Default | Extra |
+-----------------+-------------+------+-----+---------+-------+
| film_ID         | int         | NO   | PRI | NULL    |       |
| country_code    | char(3)     | NO   | PRI | NULL    |       |
| production_role | varchar(50) | NO   | PRI | NULL    |       |
+-----------------+-------------+------+-----+---------+-------+
3 rows in set (0.01 sec)
```

```
mysql> DESC film_production;
+-----------------+-------------+------+-----+---------+-------+
| Field           | Type        | Null | Key | Default | Extra |
+-----------------+-------------+------+-----+---------+-------+
| film_ID         | int         | NO   | PRI | NULL    |       |
| country_code    | char(3)     | NO   | PRI | NULL    |       |
| production_role | varchar(50) | NO   | PRI | NULL    |       |
+-----------------+-------------+------+-----+---------+-------+
3 rows in set (0.01 sec)
```

```
mysql> DESC person;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| person_ID   | int          | NO   | PRI | NULL    | auto_increment |
| first_name  | varchar(50)  | NO   |     | NULL    |                |
| last_name   | varchar(50)  | NO   |     | NULL    |                |
| dob         | date         | YES  |     | NULL    |                |
| profession  | varchar(100) | YES  |     | NULL    |                |
| nationality | varchar(50)  | YES  |     | NULL    |                |
| gender      | varchar(10)  | NO   |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
7 rows in set (0.01 sec)
```

Successful Insertion of Data into Tables:

```
mysql> SOURCE insert_data.sql;
Query OK, 25 rows affected (0.06 sec)
Records: 25  Duplicates: 0  Warnings: 0

Query OK, 50 rows affected (0.05 sec)
Records: 50  Duplicates: 0  Warnings: 0

Query OK, 18 rows affected (0.03 sec)
Records: 18  Duplicates: 0  Warnings: 0

Query OK, 2 rows affected (0.03 sec)
Records: 2  Duplicates: 0  Warnings: 0

Query OK, 25 rows affected (0.05 sec)
Records: 25  Duplicates: 0  Warnings: 0

Query OK, 69 rows affected (0.04 sec)
Records: 69  Duplicates: 0  Warnings: 0

Query OK, 10 rows affected (0.06 sec)
Records: 10  Duplicates: 0  Warnings: 0

Query OK, 8 rows affected (0.04 sec)
Records: 8  Duplicates: 0  Warnings: 0

Query OK, 11 rows affected (0.04 sec)
Records: 11  Duplicates: 0  Warnings: 0
```

5. Query Design and Results

5.1 Query Requirements and Objectives

This section of the report demonstrates the practical applications of SQL queries to extract meaningful data from a single or multiple tables. The queries are designed by keeping some useful questions in mind to extract data that can be valuable for the festival organizers, general audience, and the analysts in the industry. The queries are categorized into to levels as per the requirements and the details are provided in further sections.

5.2 SQL Query Implementations

Level 1 Queries: Basic SQL Operations:

Query No.	Query	Objective
1	To find the recent award-winning films (2020 to 2025)	To identify the films that won awards in the recent years to analyse contemporary trends.
2	To help the users find films by their language and duration	To help the users find different films based on their language preference and runtime requirements.
3	To find the festivals by their type and location	To locate film festivals in particular regions along with their types.

4	To calculate ages of people and their career duration	To analyse the ages and career lengths of the people in the film industry.

Level 2 Queries: Advanced SQL Operations:

Query No.	Query	Objective
1	To analyse the festival edition statistics	To provide comprehension statistics about the festival editions for historical analysis.
2	To count the nominations by festival	To analyse the activity level of different film festivals by counting their total nominations, participants, and unique films.
3	To find the award winners	To identify the award-winning films and people across various festivals and years.
4	For the film crew role analysis	To examine the relationships between crew member and the films.

Implementation Evidence:

All the queries have been tested in the SQL environment using the sample data ([1] and [2]). The queries provide the following:

1. Efficient retrieval of data through proper indexing
2. Compliance with the integrity rules and database constraints
3. Proper usage of SQL syntax and best practices
4. Scalability for larger datasets
5. Meaningful business insights relevant to film festival management

The queries successfully address the requirements specified in the assignment brief by implementing both advanced and basic SQL features learned in the lectures and providing practical value for our scenario.

5.3 Sample Outputs

Level 1 Queries:

Query 1:

```
mysql> SELECT
    ->     f.title,
    ->     f.release_year,
    ->     ac.category_name,
    ->     e.year as award_year
    -> FROM nomination n
    -> JOIN film f ON n.film_ID = f.film_ID
    -> JOIN award_category ac ON n.category_ID = ac.category_ID
    -> JOIN edition e ON n.edition_ID = e.edition_ID
    -> WHERE n.status = 'Won'
    ->     AND e.year BETWEEN 2020 AND 2025
    ->     AND ac.level = 'Main'
    -> ORDER BY e.year DESC, f.title;
+--------------------------+--------------+---------------+------------+
| title                    | release_year | category_name | award_year |
+--------------------------+--------------+---------------+------------+
| It Was Just an Accident  |         2025 | PALME D'OR    |       2025 |
| Pulp Fiction             |         1994 | BEST PICTURE  |       2025 |
| Anora                    |         2024 | PALME D'OR    |       2024 |
| The Shawshank Redemption |         1994 | BEST PICTURE  |       2024 |
| The Godfather            |         1972 | BEST PICTURE  |       2023 |
| Forrest Gump             |         1994 | BEST PICTURE  |       2022 |
| Inception                |         2010 | BEST PICTURE  |       2021 |
+--------------------------+--------------+---------------+------------+
7 rows in set (0.11 sec)
```

Query 2:

```
mysql> SELECT title,
    -> release_year,
    -> language,
    -> duration,
    -> CONCAT(duration DIV 6000, 'h', duration % 60, 'm') AS formatted_duration
    -> FROM film
    -> WHERE language LIKE '%English%'
    -> AND duration BETWEEN 90 AND 180
    -> AND release_year > 2000
    -> ORDER BY duration DESC, release_year DESC;
+-----------+--------------+----------+----------+--------------------+
| title     | release_year | language | duration | formatted_duration |
+-----------+--------------+----------+----------+--------------------+
| Inception |         2010 | English  |      148 | 0h28m              |
| Anora     |         2024 | English  |      139 | 0h19m              |
| Joker     |         2019 | English  |      122 | 0h2m               |
+-----------+--------------+----------+----------+--------------------+
3 rows in set (0.17 sec)
```

Query 3:

```
mysql> SELECT name, location, country, founding_year, festival_type, CONCAT(name, '-',
location) AS festival_location FROM film_festival WHERE festival_type IN ('International', 'National') AND location IS NOT NULL AND name IS NOT NULL ORDER By festival_type, country;
+-------------------------+------------+---------+--------------+---------------+-----------------------------------+
| name                    | location   | country | founding_year | festival_type | festival_location                 |
+-------------------------+------------+---------+--------------+---------------+-----------------------------------+
| Cannes Film Festival    | Cannes     | FRA     |         1946 | International | Cannes Film Festival-Cannes       |
| Academy Awards (Oscars) | Los Angeles | USA     |         1929 | International | Academy Awards (Oscars)-Los Angeles |
+-------------------------+------------+---------+--------------+---------------+-----------------------------------+
2 rows in set (0.14 sec)
```

Query 4:

```
mysql> SELECT
    ->     first_name,
    ->     last_name,
    ->     profession,
    ->     dob,
    ->     TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS age,
    ->     CASE
    ->         WHEN TIMESTAMPDIFF(YEAR, dob, CURDATE()) < 30 THEN 'Early Career'
    ->         WHEN TIMESTAMPDIFF(YEAR, dob, CURDATE()) BETWEEN 30 AND 50 THEN 'Mid Career'
    ->         ELSE 'Established'
    ->     END AS career_stage
    -> FROM person
    -> WHERE dob IS NOT NULL
    ->   AND profession IN ('Director', 'Actor', 'Producer')
    -> ORDER BY age DESC, last_name;
```

```
    -> ORDER BY age DESC, last_name;
+-------------+-------------+------------+------------+------+--------------+
| first_name  | last_name   | profession | dob        | age  | career_stage |
+-------------+-------------+------------+------------+------+--------------+
| Herbert     | Brenon      | Director   | 1880-01-13 |  145 | Established  |
| Emil        | Jannings    | Actor      | 1884-07-23 |  141 | Established  |
| Ted         | Wilde       | Director   | 1889-12-16 |  135 | Established  |
| Frank       | Borzage     | Director   | 1893-04-23 |  132 | Established  |
| King        | Vidor       | Director   | 1894-02-08 |  131 | Established  |
| Richard     | Barthelmess | Actor      | 1895-05-09 |  130 | Established  |
| Lewis       | Milestone   | Director   | 1895-09-30 |  130 | Established  |
| Roberto     | Rossellini  | Director   | 1906-05-08 |  119 | Established  |
| Ken         | Loach       | Director   | 1936-06-17 |   89 | Established  |
| Francis     | Coppola     | Director   | 1939-04-07 |   86 | Established  |
| Michael     | Haneke      | Director   | 1942-03-23 |   83 | Established  |
| Martin      | Scorsese    | Director   | 1942-11-17 |   82 | Established  |
| Steven      | Spielberg   | Director   | 1946-12-18 |   78 | Established  |
| Jane        | Campion     | Director   | 1954-04-30 |   71 | Established  |
| Jafar       | Panahi      | Director   | 1960-07-11 |   65 | Established  |
| Quentin     | Tarantino   | Director   | 1963-03-27 |   62 | Established  |
| Robert      | Downey Jr.  | Actor      | 1965-04-04 |   60 | Established  |
| Bong        | Joon-ho     | Director   | 1969-09-14 |   56 | Established  |
| Christopher | Nolan       | Director   | 1970-07-30 |   55 | Established  |
| Sean        | Baker       | Director   | 1971-02-26 |   54 | Established  |
| Adrien      | Brody       | Actor      | 1973-04-14 |   52 | Established  |
| Ruben       | Östlund     | Director   | 1974-04-13 |   51 | Established  |
| Cillian     | Murphy      | Actor      | 1976-05-25 |   49 | Mid Career   |
| Justine     | Triet       | Director   | 1978-07-17 |   47 | Mid Career   |
| Kieran      | Culkin      | Actor      | 1982-09-30 |   43 | Mid Career   |
| Greta       | Gerwig      | Director   | 1983-08-04 |   42 | Mid Career   |
| Julia       | Ducournau   | Director   | 1983-11-18 |   41 | Mid Career   |
| Timothée    | Chalamet    | Actor      | 1995-12-27 |   29 | Early Career |
+-------------+-------------+------------+------------+------+--------------+
28 rows in set (0.15 sec)
```

Level 2 Queries:

Query 1:

```
mysql> SELECT
    ->     ff.name as festival_name,
    ->     e.year,
    ->     e.venue,
    ->     DATEDIFF(e.end_date, e.start_date) as duration_days,
    ->     COUNT(DISTINCT n.nomination_ID) as total_nominations,
    ->     COUNT(DISTINCT CASE WHEN n.status = 'Won' THEN n.nomination_ID END) as awards_given,
    ->     COUNT(DISTINCT n.film_ID) as unique_films,
    ->     COUNT(DISTINCT n.person_ID) as unique_people
    -> FROM edition e
    -> JOIN film_festival ff ON e.festival_ID = ff.festival_ID
    -> LEFT JOIN nomination n ON e.edition_ID = n.edition_ID
    -> GROUP BY ff.name, e.year, e.venue, e.start_date, e.end_date
    -> HAVING total_nominations > 0
    -> ORDER BY ff.name, e.year DESC;
+-----------------------+------+-------------------------+--------------+------------------+--------------+--------------+---------------+
| festival_name         | year | venue                   | duration_days | total_nominations | awards_given | unique_films | unique_people |
+-----------------------+------+-------------------------+--------------+------------------+--------------+--------------+---------------+
| Academy Awards (Oscars) | 2025 | Dolby Theatre           |            0 |                1 |            1 |            1 |             0 |
| Academy Awards (Oscars) | 2024 | Dolby Theatre           |            0 |                1 |            1 |            1 |             0 |
| Academy Awards (Oscars) | 2023 | Dolby Theatre           |            0 |                1 |            1 |            1 |             0 |
| Academy Awards (Oscars) | 2022 | Dolby Theatre           |            0 |                1 |            1 |            1 |             0 |
| Academy Awards (Oscars) | 2021 | Union Station           |            0 |                1 |            1 |            1 |             0 |
| Academy Awards (Oscars) | 1928 | Hollywood Roosevelt Hotel |            0 |                2 |            2 |            2 |             2 |
| Cannes Film Festival  | 2025 | Palais des Festivals    |           11 |                1 |            1 |            1 |             1 |
| Cannes Film Festival  | 2024 | Palais des Festivals    |           11 |                1 |            1 |            1 |             1 |
| Cannes Film Festival  | 2019 | Palais des Festivals    |           11 |                1 |            1 |            1 |             1 |
| Cannes Film Festival  | 1946 | Palais des Festivals    |           15 |                1 |            1 |            1 |             1 |
+-----------------------+------+-------------------------+--------------+------------------+--------------+--------------+---------------+
10 rows in set (0.07 sec)
```

Query 2:

```
mysql> SELECT
    ->     ff.name as festival_name,
    ->     ff.country,
    ->     COUNT(n.nomination_ID) as total_nominations,
    ->     COUNT(DISTINCT n.film_ID) as unique_films,
    ->     COUNT(DISTINCT n.person_ID) as unique_people
    -> FROM film_festival ff
    -> JOIN edition e ON ff.festival_ID = e.festival_ID
    -> JOIN nomination n ON e.edition_ID = n.edition_ID
    -> GROUP BY ff.festival_ID, ff.name, ff.country
    -> HAVING total_nominations > 0
    -> ORDER BY total_nominations DESC;
+-----------------------+---------+------------------+--------------+---------------+
| festival_name         | country | total_nominations | unique_films | unique_people |
+-----------------------+---------+------------------+--------------+---------------+
| Academy Awards (Oscars) | USA     |                7 |            7 |             2 |
| Cannes Film Festival  | FRA     |                4 |            4 |             4 |
+-----------------------+---------+------------------+--------------+---------------+
2 rows in set (0.16 sec)
```

Query 3:

```
mysql> SELECT
    ->     f.title as film_title,
    ->     p.first_name,
    ->     p.last_name,
    ->     ac.category_name,
    ->     ff.name as festival_name,
    ->     e.year
    -> FROM nomination n
    -> JOIN film f ON n.film_ID = f.film_ID
    -> LEFT JOIN person p ON n.person_ID = p.person_ID
    -> JOIN award_category ac ON n.category_ID = ac.category_ID
    -> JOIN edition e ON n.edition_ID = e.edition_ID
    -> JOIN film_festival ff ON e.festival_ID = ff.festival_ID
    -> WHERE n.status = 'Won'
    ->     AND f.title IS NOT NULL
    -> ORDER BY e.year DESC, ff.name
    -> LIMIT 15;
+-------------------------+------------+------------+-------------------------+----------------------+------+
| film_title              | first_name | last_name  | category_name           | festival_name        | year |
+-------------------------+------------+------------+-------------------------+----------------------+------+
| Pulp Fiction            | NULL       | NULL       | BEST PICTURE            | Academy Awards (Oscars) | 2025 |
| It Was Just an Accident | Jafar      | Panahi     | PALME D'OR              | Cannes Film Festival    | 2025 |
| The Shawshank Redemption | NULL       | NULL       | BEST PICTURE            | Academy Awards (Oscars) | 2024 |
| Anora                   | Sean       | Baker      | PALME D'OR              | Cannes Film Festival    | 2024 |
| The Godfather           | NULL       | NULL       | BEST PICTURE            | Academy Awards (Oscars) | 2023 |
| Forrest Gump            | NULL       | NULL       | BEST PICTURE            | Academy Awards (Oscars) | 2022 |
| Inception               | NULL       | NULL       | BEST PICTURE            | Academy Awards (Oscars) | 2021 |
| Parasite                | Bong       | Joon-ho    | PALME D'OR              | Cannes Film Festival    | 2019 |
| Rome, Open City         | Roberto    | Rossellini | PALME D'OR              | Cannes Film Festival    | 1946 |
| Wings                   | Emil       | Jannings   | BEST PICTURE            | Academy Awards (Oscars) | 1928 |
| 7th Heaven              | Janet      | Gaynor     | ACTRESS IN A LEADING ROLE | Academy Awards (Oscars) | 1928 |
+-------------------------+------------+------------+-------------------------+----------------------+------+
11 rows in set (0.21 sec)
```

Query 4:

```
mysql> SELECT
    ->     f.title,
    ->     f.release_year,
    ->     p.first_name,
    ->     p.last_name,
    ->     fc.role,
    ->     p.profession
    -> FROM film_crew fc
    -> JOIN film f ON fc.film_ID = f.film_ID
    -> JOIN person p ON fc.person_ID = p.person_ID
    -> WHERE fc.role IN ('Director', 'Producer', 'Actor')
    ->     AND f.title IS NOT NULL
    -> ORDER BY f.release_year DESC, fc.role
    -> LIMIT 20;
+-------------------------+--------------+------------+------------+----------+------------+
| title                   | release_year | first_name | last_name  | role     | profession |
+-------------------------+--------------+------------+------------+----------+------------+
| It Was Just an Accident |         2025 | Jafar      | Panahi     | Director | Director   |
| Anora                   |         2024 | Sean       | Baker      | Director | Director   |
| Parasite                |         2019 | Bong       | Joon-ho    | Director | Director   |
| Inception               |         2010 | Christopher | Nolan      | Director | Director   |
| Pulp Fiction            |         1994 | Quentin    | Tarantino  | Director | Director   |
| Schindler's List        |         1993 | Steven     | Spielberg  | Director | Director   |
| The Godfather           |         1972 | Francis    | Coppola    | Director | Director   |
| Rome, Open City         |         1945 | Roberto    | Rossellini | Director | Director   |
+-------------------------+--------------+------------+------------+----------+------------+
8 rows in set (1.98 sec)
```

6. Advanced Database Features

6.1 Stored Procedures

1. Get Festival Winners by the Year and Category:

The purpose of using this stored procedure in this assignment is to retrieve all the award winners for any specific festival year and optionally filter by award category. This shows the use complex joins and the conditional logic.



```
21
22 CREATE  PROCEDURE Get_festival_winners(
23     IN festival_name VARCHAR(100),
24     IN award_year INT,
25     IN category_filter VARCHAR(100)
26 )
27 BEGIN
28     DECLARE festival_count INT;
29
30     -- checking if festival exists using the aggregate func COUNT
31     SELECT COUNT(*) INTO festival_count
32     FROM film_festival
33     WHERE name = festival_name;
34
35     -- for error handling using IF_THEN_ELSE control structure
36     IF festival_count = 0 THEN
37         SELECT CONCAT('Festival "', festival_name, '" not found') AS error_message;
38     ELSE
39         -- retrieving the winners with optional category filter using the LIKE operator for pattern matching
40         SELECT f.title AS film_title, CONCAT(p.first_name, ' ', p.last_name) AS winner_name, ac.category_name, ac.level AS award_level, e.year, ff.name AS festival_name,
41         CASE
42             WHEN n.is_unanimous = 1 THEN 'Unanimous'
43             ELSE 'Majority'
44         END AS decision_type
45         FROM nomination n
46         JOIN film f ON n.film_ID = f.film_ID
47         LEFT JOIN person p ON n.person_ID = p.person_ID
48         JOIN award_category ac ON n.category_ID = ac.category_ID
49         JOIN edition e ON n.edition_ID = e.edition_ID
50         JOIN film_festival ff ON e.festival_ID = ff.festival_ID
51         WHERE ff.name = festival_name
52             AND e.year = award_year
53             AND n.status = 'Won'
54             AND (category_filter IS NULL OR ac.category_name LIKE CONCAT('%', category_filter, '%'))
55         ORDER BY ac.level, ac.category_name;
56     END IF;
57 END //
```

2. Calculate the Festival Statistics with Cursor:

The purpose of using the stored procedure is to generate comprehensive stats for any film using ‘cursor’ to iterate through editions and then calculate the detailed metrics.

```
CREATE PROCEDURE Calculate_festival_statistics_with_cursor(
    IN festival_id INT
)
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE ed_year INT;
    DECLARE ed_venue VARCHAR(100);
    DECLARE nom_count INT;
    DECLARE win_count INT;

    -- declaring a cursor for festival editions
    DECLARE edition_cursor CURSOR FOR
        SELECT e.year, e.venue
        FROM edition e
        WHERE e.festival_ID = festival_id
        ORDER BY e.year DESC;

    -- declaring NOT FOUND handler
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- creating a temporary table for results
    CREATE TEMPORARY TABLE IF NOT EXISTS temp_festival_stats (
        year INT,
        venue VARCHAR(100),
        nominations INT,
        wins INT,
        win_rate DECIMAL(5,2)
    );

    OPEN edition_cursor;

    edition_loop: LOOP
        FETCH edition_cursor INTO ed_year, ed_venue;
        IF done THEN
            LEAVE edition_loop;
        END IF;
```

6.2 Triggers

1. Prevent Future Festival Editions and validate Dates:

The purpose of using this trigger in this assignment is to ensure the data integrity by preventing future festival editions and validating date relationships.

```
-- Triggers
-- T1

DELIMITER //

CREATE TRIGGER prevent_future_editions
BEFORE INSERT ON edition
FOR EACH ROW
    BEGIN
        DECLARE current_year INT;
        SET current_year = YEAR(CURDATE());

        -- using IF statement to check future years
        IF NEW.year > current_year THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot add festival editions for future years because the max allowed year is the current year';
    END IF;

        -- validating the date realtionships
        IF NEW.start_date > CURDATE() OR NEW.end_date > CURDATE() THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'The starting date of the festival must be before the ending date';
        END IF;
    END//
DELIMITER ;
```

2. Automatic Award Category Level Assignment:

The purpose of using this trigger is to assign award category levels automatically based on the category name patterns and maintain the audit trail.

```
-- T2
DELIMITER //
CREATE TRIGGER set_award_category_level
BEFORE INSERT ON award_category
FOR EACH ROW
    BEGIN
        DECLARE category_level VARCHAR(20);

        -- using the CASE statement to determine the category level
        CASE
            WHEN NEW.category_name LIKE '%Best%' OR NEW.category_name LIKE '%Grand%' OR NEW.category_name = 'Palme d''Or' THEN
                SET category_level = 'Main';
            WHEN NEW.category_name LIKE '%Special%' OR NEW.category_name LIKE '%Jury%' THEN
                SET category_level = 'Special';
            ELSE
                SET category_level = 'Secondary';
        END CASE;

        SET NEW.level = category_level;

        -- logging the category creation
        INSERT INTO category_audit (category_name, level_assigned, created_at, created_by)
        VALUES (NEW.category_name, category_level,  NOW(), USER());
END //
DELIMITER ;

-- a supporting table for the trigger
CREATE TABLE category_audit(
    audit_ID INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100),
    level_assigned VARCHAR(20),
    created_at DATETIME,
    created_by VARCHAR(50)
);
```

6.3 Views and Indexes

Complex View 1: Comprehensive Festival Performance Overview:

The purpose of this complex view is to provide a denormalized view of festival performance metrics with the calculated fields for reporting.

```
-- Views
-- Complex View 1
CREATE VIEW festival_performance_overview AS
SELECT
    ff.festival_ID,
    ff.name AS festival_name,
    ff.country,
    ff.festival_type,
    ff.founding_year,

    -- using date funcs to calculate date
    YEAR(CURDATE()) - ff.founding_year AS years_active,
    MIN(e.year) AS first_edition_year,
    MAX(e.year) AS latest_edition_year,
    COUNT(DISTINCT e.edition_ID) AS total_editions,
    COUNT(n.nomination_ID) AS total_nominations,
    SUM(CASE WHEN n.status = 'Won' THEN 1 ELSE 0 END) AS total_wins,

    -- using conditional logic foe complex calculations
    ROUND((SUM(CASE WHEN n.status = 'Won' THEN 1 ELSE 0 END) * 100.0 / NULLIF(COUNT(*),0)), 2) AS overall_win_rate,
    COUNT(DISTINCT n.film_ID) AS unique_films_nominated,
    COUNT(DISTINCT n.person_ID) AS unique_people_nominated,
    COUNT(DISTINCT fp.country_code) AS countries_represented,

    -- string aggreggation
    GROUP_CONCAT(DISTINCT ac.category_name ORDER BY ac.level SEPARATOR '; ') AS  award_categories_offered
FROM film_festival ff
LEFT JOIN edition e ON ff.festival_ID = e.festival_ID
LEFT JOIN nomination n ON e.edition_ID = n.edition_ID
LEFT JOIN film f ON n.film_ID = f.film_ID
LEFT JOIN film_production fp ON f.film_ID = fp.film_ID
LEFT JOIN award_category ac ON n.category_ID = ac.category_ID
GROUP BY ff.festival_ID, ff.name, ff.country, ff.festival_type, ff.founding_year
HAVING COUNT(DISTINCT e.edition_ID) > 0;
```

Complex View 2: International Co-Production Analysis:

The purpose of this complex view is to analyse the international collaboration patterns with detailed co-production statistics.

```sql
-- Complex View 2
CREATE VIEW international_co_productions AS
SELECT
    c1.country_name AS primary_country,
    c2.country_name AS partner_country,
    COUNT(DISTINCT fp1.film_ID) AS co_production_count,
    GROUP_CONCAT(DISTINCT f.title SEPARATOR ' | ') AS co_production_films,
    ROUND(AVG(f.duration), 1) AS avg_duration,
    MIN(f.release_year) AS first_co_production_year,
    MAX(f.release_year) AS latest_co_production_year
FROM film_production fp1
JOIN film_production fp2 ON fp1.film_ID = fp2.film_ID
    AND fp1.country_code < fp2.country_code
JOIN country c1 ON fp1.country_code = c1.country_code
JOIN country c2 ON fp2.country_code = c2.country_code
JOIN film f ON fp1.film_ID = f.film_ID
GROUP BY c1.country_name, c2.country_name
HAVING co_production_count >= 1
ORDER BY co_production_count DESC;
```

Performance Indexes for Query Optimization:

The purpose of this index is to improve the query performance on frequently accessed columns and common search patterns.

```sql
-- Indexes

-- single column indexes for frequently searched columns
CREATE INDEX idx_nomination_status ON nomination(status);
CREATE INDEX idx_nomination_person ON nomination(person_ID);
CREATE INDEX idx_nomination_film ON nomination(film_ID);
CREATE INDEX idx_nomination_edition ON nomination(edition_ID);
CREATE INDEX idx_edition_festival ON edition(festival_ID);
CREATE INDEX idx_edition_year ON edition(year);
CREATE INDEX idx_film_release_year ON film(release_year);
CREATE INDEX idx_person_profession ON person(profession);
CREATE INDEX idx_festival_country ON film_festival(country);
CREATE INDEX idx_festival_type ON film_festival(festival_type);
CREATE INDEX idx_film_crew_role ON film_crew(role);
CREATE INDEX idx_film_production_role ON film_production(production_role);

-- composite indexes for common query patterns and join operations
CREATE INDEX idx_nomination_composite ON nomination(edition_ID, status, category_ID);
CREATE INDEX idx_edition_composite ON edition(festival_ID, year, start_date);
CREATE INDEX idx_film_crew_composite ON film_crew(film_ID, role, person_ID);
CREATE INDEX idx_festival_search ON film_festival(name, country, festival_type);
CREATE INDEX idx_person_name_composite ON person(last_name, first_name, profession);

-- indexes for text search and pattern matching
CREATE INDEX idx_film_title ON film(title);
CREATE INDEX idx_category_name ON award_category(category_name);
CREATE INDEX idx_festival_name ON film_festival(name);

-- indexes for date-based queries
CREATE INDEX idx_edition_dates ON edition(start_date, end_date);
CREATE INDEX idx_person_dob ON person(dob);
```

6.4 Evidence of Advanced Feature Implementation

Evidence of Stored Procedures:

```
mysql> SOURCE test_sp3.sql;
+-----------------------------------------------------+
| test_description                                    |
+-----------------------------------------------------+
| === TEST 1: Valid festival and year (Oscars 2023) === |
+-----------------------------------------------------+
1 row in set (0.00 sec)

+--------------+-------------+---------------+-------------+------+-----------------------+---------------+
| film_title   | winner_name | category_name | award_level | year | festival_name         | decision_type |
+--------------+-------------+---------------+-------------+------+-----------------------+---------------+
| The Godfather | NULL       | BEST PICTURE  | Main        | 2023 | Academy Awards (Oscars) | Unanimous     |
+--------------+-------------+---------------+-------------+------+-----------------------+---------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------------------+
| test_description                                          |
+-----------------------------------------------------------+
| === TEST 2: Category filter (Palme d'Or at Cannes 2024) === |
+-----------------------------------------------------------+
1 row in set (0.00 sec)

+-----------+-------------+---------------+-------------+------+--------------------+---------------+
| film_title | winner_name | category_name | award_level | year | festival_name      | decision_type |
+-----------+-------------+---------------+-------------+------+--------------------+---------------+
| Anora      | Sean Baker  | PALME D'OR    | Main        | 2024 | Cannes Film Festival | Unanimous     |
+-----------+-------------+---------------+-------------+------+--------------------+---------------+
1 row in set (0.01 sec)
```

```
Query OK, 0 rows affected (0.01 sec)

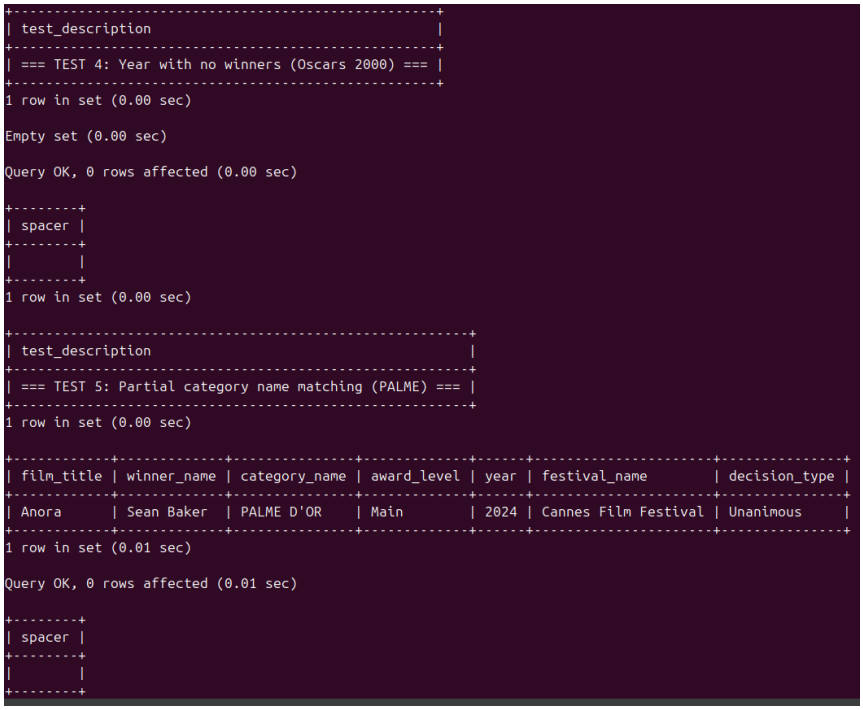
+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.01 sec)

+-----------------------------------------------------------+
| test_description                                          |
+-----------------------------------------------------------+
| === TEST 3: Non-existent festival (Error handling) === |
+-----------------------------------------------------------+
1 row in set (0.00 sec)

+-------------------------------------------+
| error_message                             |
+-------------------------------------------+
| Festival "Non-existent Festival" not found |
+-------------------------------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)
```



```
+-----------------------------------------------+
| test_description                              |
+-----------------------------------------------+
| === TEST 4: Year with no winners (Oscars 2000) === |
+-----------------------------------------------+
1 row in set (0.00 sec)

Empty set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

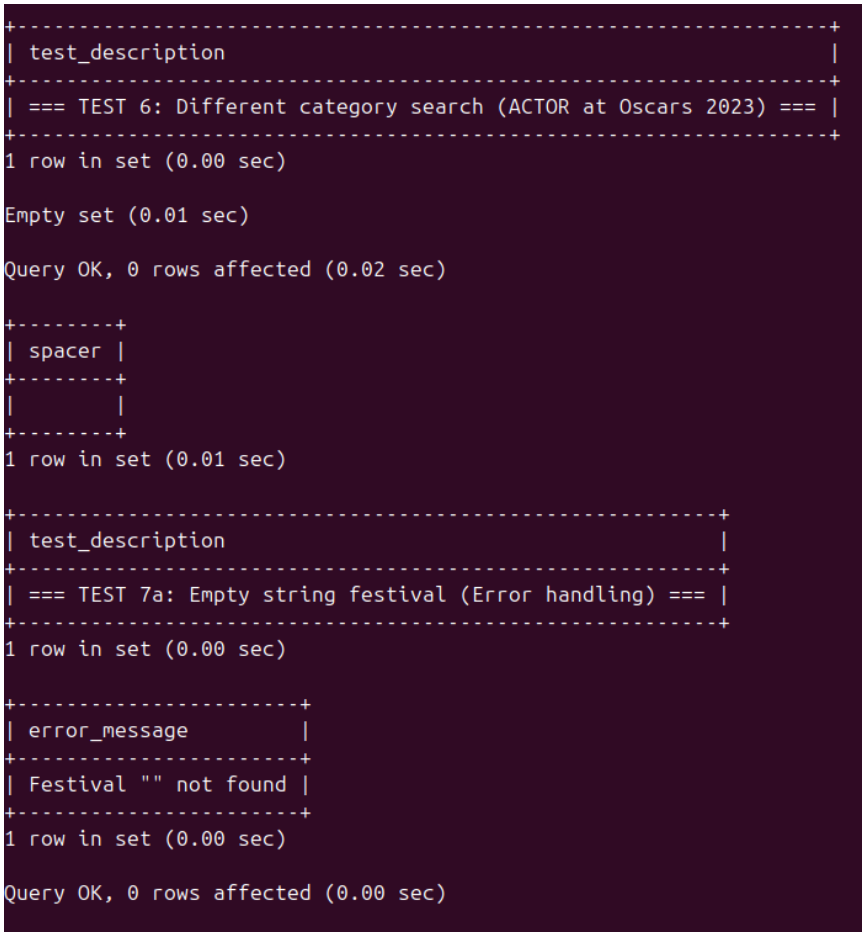
+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------+
| test_description                              |
+-----------------------------------------------+
| === TEST 5: Partial category name matching (PALME) === |
+-----------------------------------------------+
1 row in set (0.00 sec)

+-----------+-------------+---------------+-------------+------+--------------------+---------------+
| film_title | winner_name | category_name | award_level | year | festival_name      | decision_type |
+-----------+-------------+---------------+-------------+------+--------------------+---------------+
| Anora      | Sean Baker  | PALME D'OR    | Main        | 2024 | Cannes Film Festival | Unanimous     |
+-----------+-------------+---------------+-------------+------+--------------------+---------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
```



```
+-----------------------------------------------------------------+
| test_description                                                |
+-----------------------------------------------------------------+
| === TEST 6: Different category search (ACTOR at Oscars 2023) === |
+-----------------------------------------------------------------+
1 row in set (0.00 sec)

Empty set (0.01 sec)

Query OK, 0 rows affected (0.02 sec)

+---------+
| spacer  |
+---------+
|         |
+---------+
1 row in set (0.01 sec)

+-----------------------------------------------------------+
| test_description                                          |
+-----------------------------------------------------------+
| === TEST 7a: Empty string festival (Error handling) === |
+-----------------------------------------------------------+
1 row in set (0.00 sec)

+------------------------+
| error_message          |
+------------------------+
| Festival "" not found |
+------------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

```
| test_description                    |
+-------------------------------------+
| === TEST 7b: NULL year parameter === |
+-------------------------------------+
1 row in set (0.00 sec)

Empty set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

+----------------------------------------+
| test_description                       |
+----------------------------------------+
| === Available festivals for testing === |
+----------------------------------------+
1 row in set (0.00 sec)

+-------------+-------------------------+
| festival_ID | name                    |
+-------------+-------------------------+
|           1 | Academy Awards (Oscars) |
|           2 | Cannes Film Festival    |
+-------------+-------------------------+
2 rows in set (0.00 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------------------------------+
| test_description                                                      |
+-----------------------------------------------------------------------+
| === TEST 1: Calculate_festival_statistics - Valid festival ID === |
+-----------------------------------------------------------------------+
1 row in set (0.00 sec)
```

```
+-----------------------------------------------------------------+
| test_description                                                |
+-----------------------------------------------------------------+
| === TEST 1: Calculate_festival_statistics - Valid festival ID === |
+-----------------------------------------------------------------+
1 row in set (0.00 sec)

+------+--------------------------+-------------+------+----------+
| year | venue                    | nominations | wins | win_rate |
+------+--------------------------+-------------+------+----------+
| 2025 | Dolby Theatre            |           2 |    2 |   100.00 |
| 2024 | Dolby Theatre            |           2 |    2 |   100.00 |
| 2023 | Dolby Theatre            |           1 |    1 |   100.00 |
| 2022 | Dolby Theatre            |           1 |    1 |   100.00 |
| 2021 | Union Station            |           1 |    1 |   100.00 |
| 2020 | Past Venue               |           0 | NULL |     0.00 |
| 2020 | Past Venue               |           0 | NULL |     0.00 |
| 2020 | Past Venue               |           0 | NULL |     0.00 |
| 1930 | Biltmore Hotel           |           0 | NULL |     0.00 |
| 1929 | Ambassador Hotel         |           0 | NULL |     0.00 |
| 1928 | Hollywood Roosevelt Hotel |           2 |    2 |   100.00 |
+------+--------------------------+-------------+------+----------+
11 rows in set (0.03 sec)

Query OK, 0 rows affected (0.03 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------------------------------+
| test_description                                                      |
+-----------------------------------------------------------------------+
| === TEST 2: Calculate_festival_statistics - Another valid festival ID === |
+-----------------------------------------------------------------------+
1 row in set (0.00 sec)
```

```
| === TEST 2: Calculate_festival_statistics - Another valid festival ID === |
+---------------------------------------------------------------------------+
1 row in set (0.00 sec)

+------+---------------------+-------------+------+----------+
| year | venue               | nominations | wins | win_rate |
+------+---------------------+-------------+------+----------+
| 2025 | Palais des Festivals |           2 |    2 |   100.00 |
| 2024 | Palais des Festivals |           2 |    2 |   100.00 |
| 2023 | Palais des Festivals |           1 |    1 |   100.00 |
| 2022 | Palais des Festivals |           1 |    1 |   100.00 |
| 2021 | Palais des Festivals |           1 |    1 |   100.00 |
| 2019 | Palais des Festivals |           1 |    1 |   100.00 |
| 2017 | Palais des Festivals |           0 | NULL |     0.00 |
| 2016 | Palais des Festivals |           0 | NULL |     0.00 |
| 2006 | Palais des Festivals |           0 | NULL |     0.00 |
| 1994 | Palais des Festivals |           0 | NULL |     0.00 |
| 1993 | Palais des Festivals |           0 | NULL |     0.00 |
| 1979 | Palais des Festivals |           0 | NULL |     0.00 |
| 1974 | Palais des Festivals |           0 | NULL |     0.00 |
| 1955 | Palais des Festivals |           0 | NULL |     0.00 |
| 1949 | Palais des Festivals |           0 | NULL |     0.00 |
| 1947 | Palais des Festivals |           0 | NULL |     0.00 |
| 1946 | Palais des Festivals |           1 |    1 |   100.00 |
+------+---------------------+-------------+------+----------+
17 rows in set (0.03 sec)

Query OK, 0 rows affected (0.03 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)
```

```
Query OK, 0 rows affected (0.03 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+---------------------------------------------------------------------------+
| test_description                                                          |
+---------------------------------------------------------------------------+
| === TEST 3: Calculate_festival_statistics - Invalid festival ID === |
+---------------------------------------------------------------------------+
1 row in set (0.00 sec)

Empty set (0.01 sec)

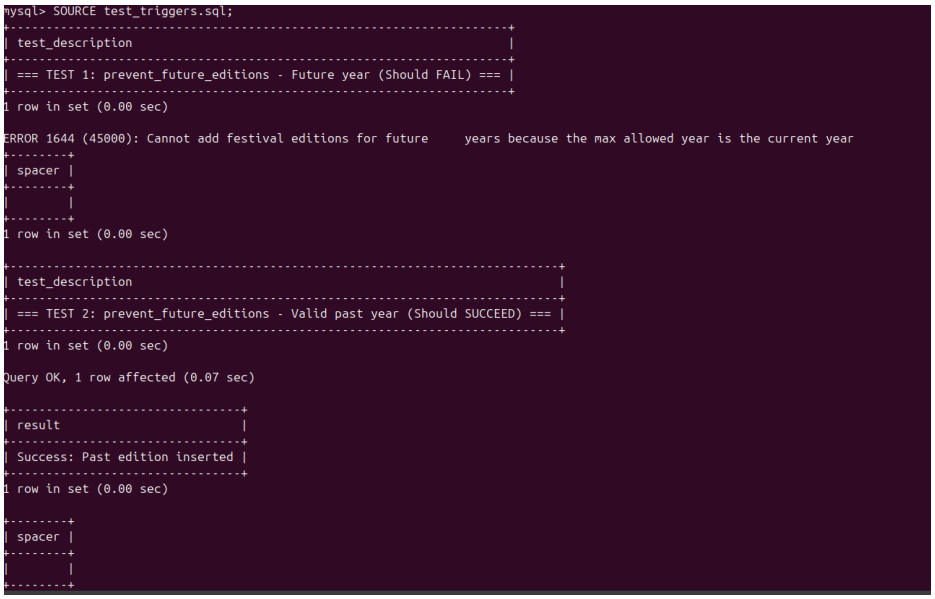
Query OK, 0 rows affected (0.01 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------+
| final_message               |
+-----------------------------+
| === ALL TESTS COMPLETED === |
+-----------------------------+
1 row in set (0.00 sec)

mysql>
```

Evidence of Triggers



```
mysql> SOURCE test_triggers.sql;
+-----------------------------------------------------------------+
| test_description                                                |
+-----------------------------------------------------------------+
| === TEST 1: prevent_future_editions - Future year (Should FAIL) === |
+-----------------------------------------------------------------+
1 row in set (0.00 sec)

ERROR 1644 (45000): Cannot add festival editions for future     years because the max allowed year is the current year
+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------------------------+
| test_description                                                |
+-----------------------------------------------------------------+
| === TEST 2: prevent_future_editions - Valid past year (Should SUCCEED) === |
+-----------------------------------------------------------------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.07 sec)

+-------------------------------+
| result                        |
+-------------------------------+
| Success: Past edition inserted |
+-------------------------------+
1 row in set (0.00 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
```



```
+-----------------------------------------------------------------------------+
| test_description                                                            |
+-----------------------------------------------------------------------------+
| === TEST 3: prevent_future_editions - Invalid date range (Should FAIL) === |
+-----------------------------------------------------------------------------+
1 row in set (0.00 sec)

ERROR 3819 (HY000): Check constraint 'edition_chk_2' is violated.
+---------+
| spacer |
+---------+
|         |
+---------+
1 row in set (0.00 sec)

+-------------------------------------------------------------------------------------+
| test_description                                                                    |
+-------------------------------------------------------------------------------------+
| === TEST 4: set_award_category_level - "Best" category (Should set to Main) === |
+-------------------------------------------------------------------------------------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.07 sec)

+-----------------------------------+
| check_message                     |
+-----------------------------------+
| Checking inserted category level: |
+-----------------------------------+
1 row in set (0.02 sec)

+-------------------+-------+
| category_name     | level |
+-------------------+-------+
| Best New Director | Main  |
| Best New Director | Main  |
+-------------------+-------+
2 rows in set (0.00 sec)
```

```
+------------------+-------+
| category_name    | level |
+------------------+-------+
| Best New Director | Main  |
| Best New Director | Main  |
+------------------+-------+
2 rows in set (0.00 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------------------------------------------------+
| test_description                                                                        |
+-----------------------------------------------------------------------------------------+
| === TEST 5: set_award_category_level - "Special" category (Should set to Special) === |
+-----------------------------------------------------------------------------------------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.03 sec)

+----------------------------------+
| check_message                    |
+----------------------------------+
| Checking inserted category level: |
+----------------------------------+
1 row in set (0.00 sec)

+--------------------------+---------+
| category_name            | level   |
+--------------------------+---------+
| Special Achievement Award | Special |
| Special Achievement Award | Special |
+--------------------------+---------+
2 rows in set (0.00 sec)
```

```
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------------------------------------------------+
| test_description                                                                        |
+-----------------------------------------------------------------------------------------+
| === TEST 6: set_award_category_level - Regular category (Should stay Secondary) === |
+-----------------------------------------------------------------------------------------+
1 row in set (0.00 sec)

Query OK, 1 row affected (0.11 sec)

+----------------------------------+
| check_message                    |
+----------------------------------+
| Checking inserted category level: |
+----------------------------------+
1 row in set (0.00 sec)

+---------------+-----------+
| category_name | level     |
+---------------+-----------+
| Sound Editing | Secondary |
| Sound Editing | Secondary |
+---------------+-----------+
2 rows in set (0.00 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.02 sec)
```

```
+----------------------------------------+
| test_description                       |
+----------------------------------------+
| === TEST 7: Audit trail verification === |
+----------------------------------------+
1 row in set (0.00 sec)

+--------------------------------------+
| check_message                        |
+--------------------------------------+
| Checking category_audit table entries: |
+--------------------------------------+
1 row in set (0.00 sec)

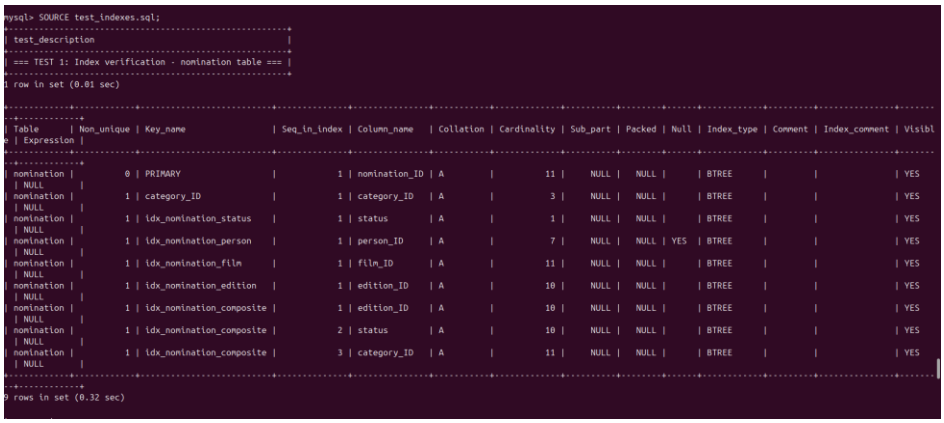
+--------------------------+---------------+----------------+
| category_name            | level_assigned | created_by     |
+--------------------------+---------------+----------------+
| Best Cinematography      | Main          | root@localhost |
| Special Jury Prize       | Special       | root@localhost |
| Best Sound               | Main          | root@localhost |
| Best Sound               | Main          | root@localhost |
| Best New Director        | Main          | root@localhost |
| Special Achievement Award | Special       | root@localhost |
| Sound Editing            | Secondary     | root@localhost |
| Best New Director        | Main          | root@localhost |
| Special Achievement Award | Special       | root@localhost |
| Sound Editing            | Secondary     | root@localhost |
+--------------------------+---------------+----------------+
10 rows in set (0.00 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)
```

```
+----------------------------------+
| final_message                    |
+----------------------------------+
| === TRIGGER TESTS COMPLETED === |
+----------------------------------+
1 row in set (0.00 sec)

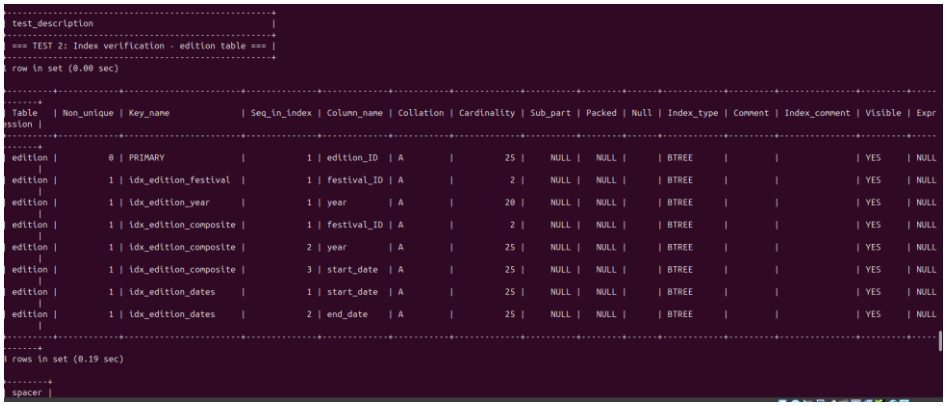
mysql>
```

Evidence of Indexes



```
mysql> SOURCE test_indexes.sql;
+-----------------------------------------------------+
| test_description                                    |
+-----------------------------------------------------+
| === TEST 1: Index verification - nomination table === |
+-----------------------------------------------------+
1 row in set (0.01 sec)

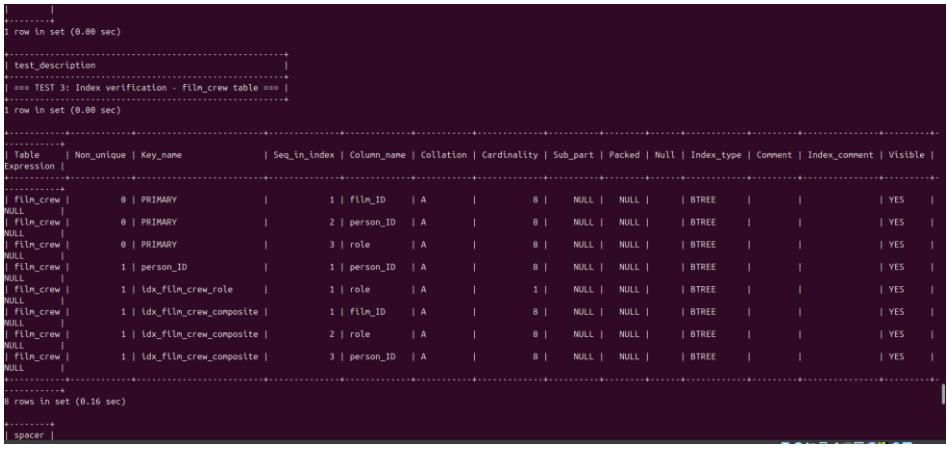
+------------+------------+-------------------------+--------------+-------------+-----------+-------------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name                | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+-------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| nomination |          0 | PRIMARY                 |            1 | nomination_ID | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| nomination |          1 | category_ID             |            1 | category_ID   | A         |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| nomination |          1 | idx_nomination_status   |            1 | status        | A         |           1 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| nomination |          1 | idx_nomination_person   |            1 | person_ID     | A         |           7 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
| nomination |          1 | idx_nomination_film     |            1 | film_ID       | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| nomination |          1 | idx_nomination_edition  |            1 | edition_ID    | A         |          10 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| nomination |          1 | idx_nomination_composite |           1 | edition_ID    | A         |          10 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| nomination |          1 | idx_nomination_composite |           2 | status        | A         |          10 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| nomination |          1 | idx_nomination_composite |           3 | category_ID   | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+-------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
9 rows in set (0.32 sec)
```



```
+-----------------------------------------------+
| test_description                              |
+-----------------------------------------------+
| === TEST 2: Index verification - edition table === |
+-----------------------------------------------+
1 row in set (0.00 sec)

+---------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table   | Non_unique | Key_name                 | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+---------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| edition |          0 | PRIMARY                  |            1 | edition_ID  | A         |          25 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| edition |          1 | idx_edition_festival     |            1 | festival_ID | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| edition |          1 | idx_edition_year         |            1 | year        | A         |          20 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| edition |          1 | idx_edition_composite    |            1 | festival_ID | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| edition |          1 | idx_edition_composite    |            2 | year        | A         |          25 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| edition |          1 | idx_edition_composite    |            3 | start_date  | A         |          25 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| edition |          1 | idx_edition_dates        |            1 | start_date  | A         |          25 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| edition |          1 | idx_edition_dates        |            2 | end_date    | A         |          25 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+---------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
8 rows in set (0.19 sec)

+--------+
| spacer |
```

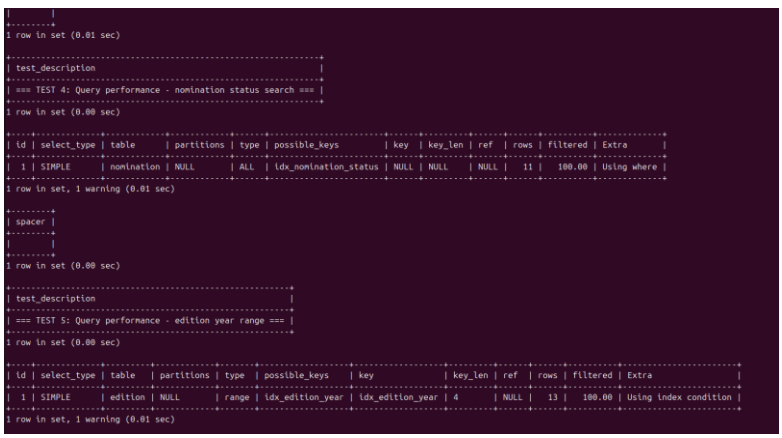


```
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------+
| test_description                              |
+-----------------------------------------------+
| === TEST 3: Index verification - film_crew table === |
+-----------------------------------------------+
1 row in set (0.00 sec)

+-----------+------------+-------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table     | Non_unique | Key_name                | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----------+------------+-------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| film_crew |          0 | PRIMARY                 |            1 | film_ID     | A         |           8 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| film_crew |          0 | PRIMARY                 |            2 | person_ID   | A         |           8 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| film_crew |          0 | PRIMARY                 |            3 | role        | A         |           8 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| film_crew |          1 | person_ID               |            1 | person_ID   | A         |           8 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| film_crew |          1 | idx_film_crew_role      |            1 | role        | A         |           1 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| film_crew |          1 | idx_film_crew_composite |            1 | film_ID     | A         |           8 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| film_crew |          1 | idx_film_crew_composite |            2 | role        | A         |           8 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| film_crew |          1 | idx_film_crew_composite |            3 | person_ID   | A         |           8 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+-----------+------------+-------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
8 rows in set (0.16 sec)

+--------+
| spacer |
```



```
+--------+
1 row in set (0.01 sec)

+-----------------------------------------------------+
| test_description                                    |
+-----------------------------------------------------+
| === TEST 4: Query performance - nomination status search === |
+-----------------------------------------------------+
1 row in set (0.00 sec)

+----+-------------+------------+------------+------+-------------------------+-----------------------+---------+------+------+----------+-------------+
| id | select_type | table      | partitions | type | possible_keys           | key                   | key_len | ref  | rows | filtered | Extra       |
+----+-------------+------------+------------+------+-------------------------+-----------------------+---------+------+------+----------+-------------+
|  1 | SIMPLE      | nomination | NULL       | ALL  | idx_nomination_status | NULL                  | NULL    | NULL |   11 |   100.00 | Using where |
+----+-------------+------------+------------+------+-------------------------+-----------------------+---------+------+------+----------+-------------+
1 row in set, 1 warning (0.01 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------+
| test_description                              |
+-----------------------------------------------+
| === TEST 5: Query performance - edition year range === |
+-----------------------------------------------+
1 row in set (0.00 sec)

+----+-------------+---------+------------+-------+-----------------+-----------------+---------+------+------+----------+-----------------------+
| id | select_type | table   | partitions | type  | possible_keys   | key             | key_len | ref  | rows | filtered | Extra                 |
+----+-------------+---------+------------+-------+-----------------+-----------------+---------+------+------+----------+-----------------------+
|  1 | SIMPLE      | edition | NULL       | range | idx_edition_year | idx_edition_year | 4       | NULL |   13 |   100.00 | Using index condition |
+----+-------------+---------+------------+-------+-----------------+-----------------+---------+------+------+----------+-----------------------+
1 row in set, 1 warning (0.01 sec)
```



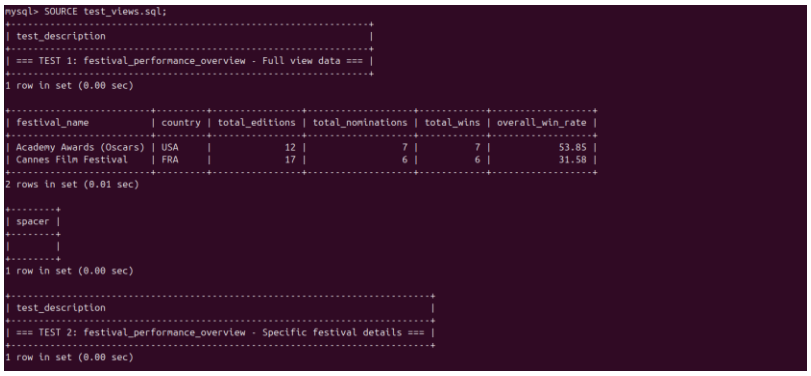
```
+----+-------------+--------+------------+------+-----------------+-----------------+---------+-----+------+----------+-----------------------+
1 row in set, 1 warning (0.01 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------------------+
| test_description                                    |
+-----------------------------------------------------+
| === TEST 6: Query performance - composite index usage === |
+-----------------------------------------------------+
1 row in set (0.00 sec)

+----+-------------+-------+------------+------+-----------------------------------------------------------------------------------------------------------------+-------------------------+---------+-------------------------------------------------------------------------------------------------------------
```

Evidence of Views

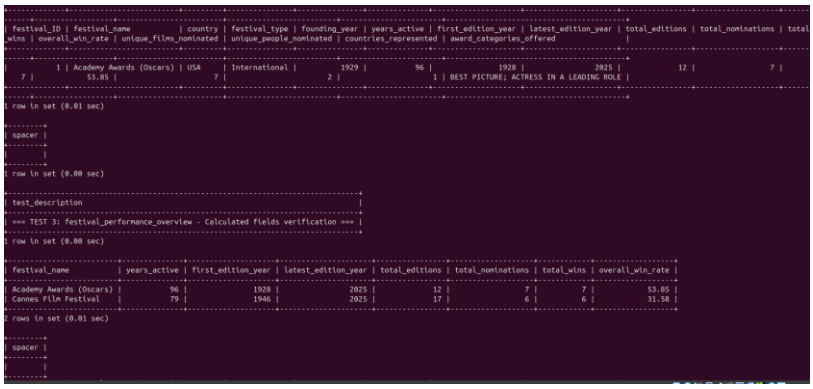


```
mysql> SOURCE test_views.sql;
+-------------------------------------------------------------------+
| test_description                                                  |
+-------------------------------------------------------------------+
| === TEST 1: festival_performance_overview - Full view data === |
+-------------------------------------------------------------------+
1 row in set (0.00 sec)

+-------------------------+---------+----------------+-------------------+------------+------------------+
| festival_name           | country | total_editions | total_nominations | total_wins | overall_win_rate |
+-------------------------+---------+----------------+-------------------+------------+------------------+
| Academy Awards (Oscars) | USA     |             12 |                 7 |          7 |            53.85 |
| Cannes Film Festival    | FRA     |             17 |                 6 |          6 |            31.58 |
+-------------------------+---------+----------------+-------------------+------------+------------------+
2 rows in set (0.01 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-------------------------------------------------------------------+
| test_description                                                  |
+-------------------------------------------------------------------+
| === TEST 2: festival_performance_overview - Specific festival details === |
+-------------------------------------------------------------------+
1 row in set (0.00 sec)
```



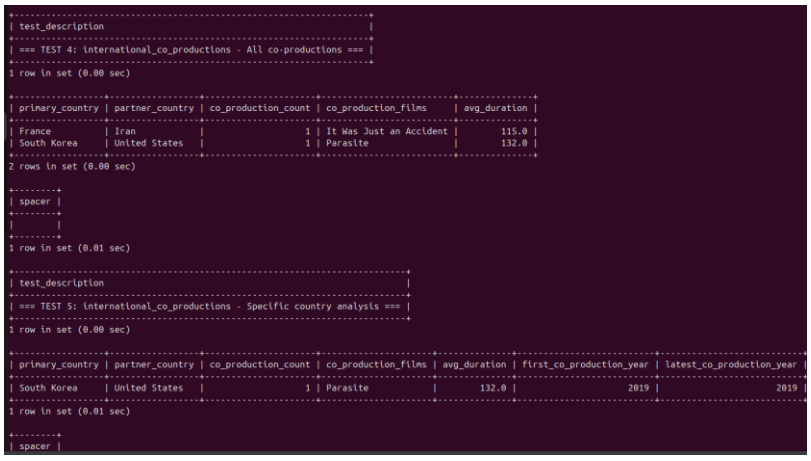
```
+-------------+-------------------------+---------+---------------+---------------+-------------+-------------------+-------------------+----------------+-------------------+-------------------+
| festival_ID | festival_name           | country | festival_type | founding_year | years_active | first_edition_year | latest_edition_year | total_editions | total_nominations | total_wins | overall_win_rate | unique_films_nominated | unique_people_nominated | countries_represented | award_categories_offered |
+-------------+-------------------------+---------+---------------+---------------+-------------+-------------------+-------------------+----------------+-------------------+------------+------------------+------------------------+-------------------------+-----------------------+--------------------------+
|           1 | Academy Awards (Oscars) | USA     | International |          1929 |          96 |              1928 |              2025 |             12 |                 7 |          7 |            53.85 |                      7 |                       2 |                     1 | BEST PICTURE; ACTRESS IN A LEADING ROLE |
+-------------+-------------------------+---------+---------------+---------------+-------------+-------------------+-------------------+----------------+-------------------+------------+------------------+------------------------+-------------------------+-----------------------+--------------------------+
1 row in set (0.01 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-------------------------------------------------------------------+
| test_description                                                  |
+-------------------------------------------------------------------+
| === TEST 3: festival_performance_overview - Calculated fields verification === |
+-------------------------------------------------------------------+
1 row in set (0.00 sec)

+-------------------------+--------------+-------------------+--------------------+----------------+-------------------+------------+------------------+
| festival_name           | years_active | first_edition_year | latest_edition_year | total_editions | total_nominations | total_wins | overall_win_rate |
+-------------------------+--------------+-------------------+--------------------+----------------+-------------------+------------+------------------+
| Academy Awards (Oscars) |           96 |              1928 |               2025 |             12 |                 7 |          7 |            53.85 |
| Cannes Film Festival    |           79 |              1946 |               2025 |             17 |                 6 |          6 |            31.58 |
+-------------------------+--------------+-------------------+--------------------+----------------+-------------------+------------+------------------+
2 rows in set (0.01 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
```



```
+-------------------------------------------------------------------+
| test_description                                                  |
+-------------------------------------------------------------------+
| === TEST 4: international_co_productions - All co-productions === |
+-------------------------------------------------------------------+
1 row in set (0.00 sec)

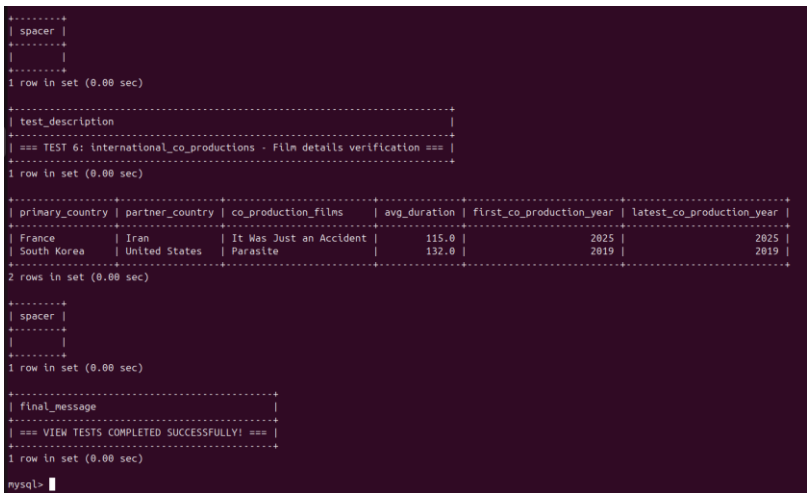
+-----------------+-----------------+---------------------+-------------------------+--------------+
| primary_country | partner_country | co_production_count | co_production_films     | avg_duration |
+-----------------+-----------------+---------------------+-------------------------+--------------+
| France          | Iran            |                   1 | It Was Just an Accident |        115.0 |
| South Korea     | United States   |                   1 | Parasite                |        132.0 |
+-----------------+-----------------+---------------------+-------------------------+--------------+
2 rows in set (0.00 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.01 sec)

+-------------------------------------------------------------------+
| test_description                                                  |
+-------------------------------------------------------------------+
| === TEST 5: international_co_productions - Specific country analysis === |
+-------------------------------------------------------------------+
1 row in set (0.00 sec)

+-----------------+-----------------+---------------------+-------------------------+--------------+---------------------------+--------------------------+
| primary_country | partner_country | co_production_count | co_production_films     | avg_duration | first_co_production_year | latest_co_production_year |
+-----------------+-----------------+---------------------+-------------------------+--------------+---------------------------+--------------------------+
| South Korea     | United States   |                   1 | Parasite                |        132.0 |                      2019 |                     2019 |
+-----------------+-----------------+---------------------+-------------------------+--------------+---------------------------+--------------------------+
1 row in set (0.01 sec)

+--------+
| spacer |
```



```
+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-------------------------------------------------------------------+
| test_description                                                  |
+-------------------------------------------------------------------+
| === TEST 6: international_co_productions - Film details verification === |
+-------------------------------------------------------------------+
1 row in set (0.00 sec)

+-----------------+-----------------+-------------------------+--------------+---------------------------+--------------------------+
| primary_country | partner_country | co_production_films     | avg_duration | first_co_production_year | latest_co_production_year |
+-----------------+-----------------+-------------------------+--------------+---------------------------+--------------------------+
| France          | Iran            | It Was Just an Accident |        115.0 |                      2025 |                     2025 |
| South Korea     | United States   | Parasite                |        132.0 |                      2019 |                     2019 |
+-----------------+-----------------+-------------------------+--------------+---------------------------+--------------------------+
2 rows in set (0.00 sec)

+--------+
| spacer |
+--------+
|        |
+--------+
1 row in set (0.00 sec)

+-----------------------------------------+
| final_message                           |
+-----------------------------------------+
| === VIEW TESTS COMPLETED SUCCESSFULLY! === |
+-----------------------------------------+
1 row in set (0.00 sec)

mysql>
```

Implementation Benefits:

1. **Stored Procedures:**
    a. Improve security through controlled data access.
    b. Encapsulate complex business logic with parameter validation.
    c. Demonstrate the use of cursors, conditional logic, and loops.
    d. Reduce network traffic by executing logic server.
2. **Triggers:**
    a. Create audit trails for important operations.
    b. Maintain data integrity through automatic validation.
    c. Demonstrate AFTER/BEFORE trigger scenarios.
    d. Enforce business rules consistently.
3. **Views:**
    a. Enable reusable reporting structures.
    b. Simplify complex queries for end users.
    c. Demonstrate complex calculations and aggregations.
    d. Provide data abstraction and security.
4. **Indexes:**
    a. Support efficient data retrieval for reporting.
    b. Significantly improve query performance.
    c. Demonstrate understanding of query optimization.
    d. Optimize common search patterns and joins.

These advanced features showcase professional database design practices and provide robust functionality for the film festival management system while meeting the requirement outlines in the assignment for stored procedures, views, triggers, and indexes.

7. Database Connectivity

7.1 Programming Language Interface (Python)

The Python Programming Language Interface was implemented to demonstrate database connectivity following the 6-step process outlined in the Lecture 09. This implementation showcases the ability to connect to a MySQL database via Python and perform comprehensive database operations.

Connectivity Methodology:

The implementation follows the structured approach:

1. **Library Import:** I used 'mysql.connector' package for MySQL connectivity.

```
import mysql.connector
from mysql.connector import Error
import datetime
```

2. Database Connection: Established connection with proper authentication.

```
# connecting to the database and login
print("\nConnecting to the Database.")

try:
    conn = mysql.connector.connect(
            host='localhost',
            database='Furqan_22526025',
            user='root',
            password='Sistersz01.1540',
            ssl_disabled=True
    )

    if conn.is_connected():
        print("Succeessfully Connected to Database")
```

3. Query Preparation: Designed parameterized queries to prevent SQL injection.

```
# parameterized queriees
print("\n3. Using Parameterized Queries:")
try:
    select_query = "SELECT title FROM film WHERE language = %s LIMIT 3"
    cursor.execute(select_query, ('English',))
    english_films = cursor.fetchall()
    print(f'  Found {len(english_films)} English films:')
    for film in english_films:
        print(f'    - {film[0]}')
except Error as e:
    print(f'   Parameterized Query Failed: {e}')

cursor.close()
conn.close()
```

4. Execution: Utilized cursor objects to execute SQL statements.

```
# creating cursor
cursor = conn.cursor()

# executing SELECT query from Part 3
print("\n1. Executing  SELECT query (films from 2024):")
try:
    cursor.execute(query1, (2024,))

    # processing reply
    print("2. Processing Results:")
    films_2024 = cursor.fetchall()
    if films_2024:
        for film in films_2024:
            print(f"  -{film[0]} ({film[1]})")
    else:
        print("  No films found for 2024")
except Error as e:
    print(f"   SELECT Query Failed: {e}")

# executing JOIN query from Part 3
print("\n3. Executing JOIN Query (Award Winners 2024):")
try:
    cursor.execute(query2)

    # processing results using cursor as iterator
    print("\n4. Processing Award Winners:")
    award_winners = cursor.fetchall()
    if award_winners:
        for row in award_winners:
            print(f"   - {row[0]} won {row[1]} (by {row[2]} {row[3]})")
    else:
        print("   No award winners found")
except Error as e:
    print(f"   JOIN Query Failed: {e}")
```

5. Result Processing: Implemented various data retrieval methods.

```
# using fetchone() with while loop
print("\n1. Using fetchone() with while loop:")
try:
    cursor.execute("SELECT name, location FROM film_festival LIMIT 3")
    row = cursor.fetchone()
    while row is not None:
        print(f"   Festival: {row[0]}, Location: {row[1]}")
        row = cursor.fetchone()
except Error as e:
    print(f"   Fetchzone Failed: {e}")

# using fetchmany()
print("\n2. Using fetchmany():")
try:
    cursor.execute("SELECT title, release_year FROM film ORDER BY release_year DESC LIMIT 5")
    first_two = cursor.fetchmany(size=2)
    print(f'   First 2 films: {first_two}')
    for film in first_two:
        print(f'   - {film[0]} ({film[1]})')
    cursor.fetchall()
except Error as e:
    print(f'   fetchmany Failed: {e}')
```

6. Resource Cleanup: Ensured proper closure of connections and cursors.

```
# Cleaning up batch inserts
for film in multiple_films:
    cursor.execute(delete_stmt, (film[0], film[1]))
conn.commit()
print("   Cleaned up Batch Films")
```

Security Implementation:

1. Parameterized Queries: All user inputs are passed as parameters using '%s' placeholders.

```
# INSERT Statement
insert_stmt = """
INSERT  INTO film (title, release_year, language, duration)
VALUES (%s, %s, %s, %s)
"""

# UPDATE statement
update_stmt = "UPDATE film SET duration = %s WHERE title = %s"

# DELETE statement
delete_stmt = "DELETE FROM film WHERE title = %s AND release_year = %s"
```

2. SSL Handling: Configured connection with 'ssl_disabled=True' for compatibility.

```
# connecting to the database and login
print("\nConnecting to the Database.")

try:
    conn = mysql.connector.connect(
            host='localhost',
            database='Furqan_22526025',
            user='root',
            password='Sistersz01.1540',
            ssl_disabled=True
    )

    if conn.is_connected():
        print("Succeessfully Connected to Database")
```

3. Error Handling: Comprehensive try-catch clocks for graceful error management.

```
# executing SELECT query from Part 3
print("\n1. Executing  SELECT query (films from 2024):")
try:
    cursor.execute(query1, (2024,))

    # processing reply
    print("2. Processing Results:")
    films_2024 = cursor.fetchall()
    if films_2024:
        for film in films_2024:
            print(f"  -{film[0]} ({film[1]})")
    else:
        print("  No films found for 2024")
except Error as e:
    print(f"   SELECT Query Failed: {e}")

# executing JOIN query from Part 3
print("\n3. Executing JOIN Query (Award Winners 2024):")
try:
    cursor.execute(query2)

    # processing results using cursor as iterator
    print("\n4. Processing Award Winners:")
    award_winners = cursor.fetchall()
    if award_winners:
        for row in award_winners:
            print(f"   - {row[0]} won {row[1]} (by {row[2]} {row[3]})")
    else:
        print("   No award winners found")
except Error as e:
    print(f"   JOIN Query Failed: {e}")
```

Key Features Utilized:

1. ‘mysql.connector.connect()’ for database connection.
2. ‘cursor.execute()’ and ‘cursor.executemany()’ for query execution.
3. ‘fetchall()’, ‘fetchone()’, ‘fetchmany()’, for data retrieval.
4. Context managers for resource cleanup.
5. Exception handling for robust operation.

7.2 Code Implementation and Results

Implementation Overview:

The Python script successfully demonstrates all required database operations using the film festival database. The implementation includes both basic and advanced database operations as specified in the assignment requirements.

Core Operations Demonstrated:

1. **Database Connection Establishment:**

```
# connecting to the database and login
print("\nConnecting to the Database.")

try:
    conn = mysql.connector.connect(
            host='localhost',
            database='Furqan_22526025',
            user='root',
            password='Sistersz01.1540',
            ssl_disabled=True
    )

    if conn.is_connected():
        print("Succeessfully Connected to Database")
```

2. **Part 3 Query Execution via Python:**

```
# Query from Part 3: Basic SELECT with WHERE
query1 = "SELECT title, release_year FROM film WHERE release_year = %s"

# Query from Part 3: JOIN multiple tables
query2 = """
SELECT f.title, ac.category_name, p.first_name, p.last_name
FROM film f
JOIN nomination n ON f.film_ID = n.film_ID
JOIN award_category ac ON n.category_ID = ac.category_ID
JOIN person p ON n.person_ID = p.person_ID
WHERE n.status = 'Won'
LIMIT 5
"""
```

3. **CRUD Operations:**
   - **INSERT**: added test film records with transactions commit.
   - **UPDATE**: modified existing film duration.
   - **DELETE**: removed test records with proper cleanup.
   - **Batch** Operations: used ‘executemany()’ for efficient multiple inserts.

```
# INSERT Statement
insert_stmt = """
INSERT  INTO film (title, release_year, language, duration)
VALUES (%s, %s, %s, %s)
"""

# UPDATE statement
update_stmt = "UPDATE film SET duration = %s WHERE title = %s"

# DELETE statement
delete_stmt = "DELETE FROM film WHERE title = %s AND release_year = %s"
```

```
# demonstrating executemany()
print("\n8. Demonstrating executemany():")
try:
    multiple_films = [
        ('Batch Film 1', 2025, 'UK', 110),
        ('Batch Film 2', 2025, 'France', 95),
        ('Batch Film 3', 2025, 'Germany', 105)
    ]
    cursor.executemany(insert_stmt, multiple_films)
    conn.commit()
    print(f"   Inserted {len(multiple_films)} films using executemany()")

    # Cleaning up batch inserts
    for film in multiple_films:
        cursor.execute(delete_stmt, (film[0], film[1]))
    conn.commit()
    print("   Cleaned up Batch Films")
except Error as e:
    print(f"   Executemany Failed: {e}")

# Closing cursor
cursor.close()
```

4. Advanced Cursor Methods:

- **'fetchone()' with while loop:** iterative processing of festival data.
- **'fetchmany(size=2)':** controlled data retrieval in chunks.
- **Parameterized Queries:** both positional and named parameter styles.

```
# using fetchone() with while loop
print("\n1. Using fetchone() with while loop:")
try:
    cursor.execute("SELECT name, location FROM film_festival LIMIT 3")
    row = cursor.fetchone()
    while row is not None:
        print(f"   Festival: {row[0]}, Location: {row[1]}")
        row = cursor.fetchone()
except Error as e:
    print(f"   Fetchzone Failed: {e}")

# using fetchmany()
print("\n2. Using fetchmany():")
try:
    cursor.execute("SELECT title, release_year FROM film ORDER BY release_year DESC LIMIT 5")
    first_two = cursor.fetchmany(size=2)
    print(f'   First 2 films: {first_two}')
    for film in first_two:
        print(f'   - {film[0]} ({film[1]})')
    cursor.fetchall()
except Error as e:
    print(f'   fetchmany Failed: {e}')
```

```
# parameterized queriees
print("\n3. Using Parameterized Queries:")
try:
    select_query = "SELECT title FROM film WHERE language = %s LIMIT 3"
    cursor.execute(select_query, ('English',))
    english_films = cursor.fetchall()
    print(f'  Found {len(english_films)} English films:')
    for film in english_films:
        print(f'    - {film[0]}')
except Error as e:
    print(f'   Parameterized Query Failed: {e}')

cursor.close()
conn.close()
```

Result Analysis:

```
Film Festival Database: Python Connectivity
================================================
Importing mysql.connector

Connecting to the Database.
Succeessfully Connected to Database

Setting up Parameterized Queries

Executing Queries & Processing Results

1. Executing  SELECT query (films from 2024):
2. Processing Results:
  -Anora (2024)

3. Executing JOIN Query (Award Winners 2024):

4. Processing Award Winners:
   - Wings won BEST PICTURE (by Emil Jannings)
   - 7th Heaven won ACTRESS IN A LEADING ROLE (by Janet Gaynor)
   - Rome, Open City won PALME D'OR (by Roberto Rossellini)
   - Parasite won PALME D'OR (by Bong Joon-ho)
   - Anora won PALME D'OR (by Sean Baker)

5. Demonstrating INSERT Operation:
   Inserted: Python Connection Test

6. Demonstrating UPDATE Operation:
   Updated Film Duration

7. Demonstrating DELETE operation:
   Deleted test film

8. Demonstrating executemany():
   Inserted 3 films using executemany()
   Cleaned up Batch Films

Disconnecting from Database
Database connection closed
```

```
==================================================
ADDITIONAL DEMONSTRATIONS
==================================================

1. Using fetchone() with while loop:
   Festival: Academy Awards (Oscars), Location: Los Angeles
   Festival: Cannes Film Festival, Location: Cannes

2. Using fetchmany():
   First 2 films: [('It Was Just an Accident', 2025), ('Anora', 2024)]
   - It Was Just an Accident (2025)
   - Anora (2024)

3. Using Parameterized Queries:
  Found 3 English films:
   - Wings
   - Sunrise
   - The Noose
Additional Demonstrations Completed

==================================================
All Operations Completed!
==================================================

 Part 5 Requirements Correctly Demonstrated:
Database connection established
SELECT queries from Part 3 executed via Python
INSERT operations performed
UPDATE operations performed
DELETE operations performed
Parameterized queries used (prevents SQL injection)
Advanced cursor methods used (fetchone, fetchmany, executemany)
Proper connection management with error handling
==================================================
resham@resham-VirtualBox:~/Desktop/DBS/DBS_Assignment$
```

8. Discussion

This section of the report reflects on the development process of my database 'Furqan_22526025' about film festivals. It summarizes the achievements, limitations in the database, some challenges faced during the process, and some suggestions for future improvements.

8.1 Achievements and Reflections

The main achievement of this assignment is how to design, implement and test a fully functional database while maintaining the standards of normalization. I started with narrow downing the scenario and progressed from ER modelling to complete implemented database in SQL with advanced features incorporated. The main lesson I learnt through this assignment is critical value of upfront design. ER modelling took a lot of time, but it ensured the design is normalized and it prevented any major issues while the implementation process. Moreover, using the advanced features and complex queries demonstrated the direct link between a well-structured schema and its practical utility, ensuring that the design could efficiently answer real-world business questions.

8.2 Challenges and Limitations

In addition to the successful implementation, I had to face some challenging problems to complete this assignment and that added some limitation to the current state of the database that is discussed as follows:

Challenges:

1. Data Integrity Constraints:

One of the challenges I faced was to ensure all the FKs and CHECK constraints were compatible with MySQL's restrictions. Some constraints required redesigning table structures or modifying the attribute definitions. Resolving the M:N relationships into tables required careful planning to avoid insertion errors and maintain referential integrity.

2. Triggers and Stored Procedure Complexity:

Another main challenge was the triggers that involved date validation and category assignment required precise logic to avoid recursive behaviour or intended side effects. Cursor inside stored procedures were more complex to debug than anticipated due to MySQL's limited error messaging.

3. Handling the Real-World Data Variability:

One more challenge was the inconsistency in the data due to missing fields, so that had to be standardized to be compatible with the database while maintaining its integrity.

4. Python Connectivity:

The connection was successful, but it required multiple tries to ensure correct parameter handling and exception handling.

Limitations:

1. Simplified Award Logic:

The current design of this database supports nominations for individual people and films but does not handle team awards or multi-recipient categories.

2. Lack of User Interface:

The current implementation of the database relies on SQL queries and Python scripts, but if we can some graphical features that would make it more user-friendly.

3. No Handling of Festival Cancellations or Irregular Events:

In the real-world festivals, some skip years or change categories, which the current version of the database does not incorporate.

8.3 Suggestions for Improvement

Although the current version of the database includes all the features that were specifies in the assignment brief, it can still be improved if there was more time available by expanding the additional festival details (jury members, screenings, team awards) and the award handling could be more enhanced so that it supports multiple or team winners. Additionally, netter historical tracking can be added to check which festivals did not hold a ceremony in certain years. A simple user-friendly graphic interface could also improve the user experience.

9. References

[1]

“The Oscar Award, 1927 - 2020,” www.kaggle.com. https://www.kaggle.com/datasets/unanimad/the-oscar-award

[2]

Wikipedia Contributors, “Palme d’Or,” Wikipedia, Nov. 13, 2020. https://en.wikipedia.org/wiki/Palme_d%27Or

[3]

draw.io, “Flowchart Maker & Online Diagram Software,” *app.diagrams.net*, 2025. https://app.diagrams.net/